

Whether you choose Artemis or ActiveMQ, you shouldn't need to configure these properties for development when the broker is running locally.

If you're using ActiveMQ, you will, however, need to set the `spring.activemq.in-memory` property to `false` to prevent Spring from starting an in-memory broker. An in-memory broker may seem useful, but it's helpful only when you'll be consuming messages from the same application that publishes them (which has limited usefulness).

Instead of using an embedded broker, you'll want to install and start an Artemis (or ActiveMQ) broker before moving on. Rather than repeat the installation instructions here, I refer you to the broker documentation for details:

- *Artemis*—<http://mng.bz/yJOo>
- *ActiveMQ*—<http://mng.bz/MveD>

With the JMS starter in your build and a broker waiting to ferry messages from one application to another, you're ready to start sending messages.

9.1.2 *Sending messages with JmsTemplate*

With a JMS starter dependency (either Artemis or ActiveMQ) in your build, Spring Boot will autoconfigure a `JmsTemplate` (among other things) that you can inject and use to send and receive messages.

`JmsTemplate` is the centerpiece of Spring's JMS integration support. Much like Spring's other template-oriented components, `JmsTemplate` eliminates a lot of boilerplate code that would otherwise be required to work with JMS. Without `JmsTemplate`, you'd need to write code to create a connection and session with the message broker and more code to deal with any exceptions that might be thrown in the course of sending a message. `JmsTemplate` focuses on what you really want to do: send a message.

`JmsTemplate` has several methods that are useful for sending messages, including the following:

```
// Send raw messages
void send(MessageCreator messageCreator) throws JmsException;
void send(Destination destination, MessageCreator messageCreator)
    throws JmsException;
void send(String destinationName, MessageCreator messageCreator)
    throws JmsException;

// Send messages converted from objects
void convertAndSend(Object message) throws JmsException;
void convertAndSend(Destination destination, Object message)
    throws JmsException;
void convertAndSend(String destinationName, Object message)
    throws JmsException;

// Send messages converted from objects with post-processing
void convertAndSend(Object message,
    MessagePostProcessor postProcessor) throws JmsException;
void convertAndSend(Destination destination, Object message,
    MessagePostProcessor postProcessor) throws JmsException;
void convertAndSend(String destinationName, Object message,
    MessagePostProcessor postProcessor) throws JmsException;
```

As you can see, there are really only two methods, `send()` and `convertAndSend()`, each overridden to support different parameters. And if you look closer, you'll notice that the various forms of `convertAndSend()` can be broken into two subcategories. In trying to understand what all of these methods do, consider the following breakdown:

- Three `send()` methods require a `MessageCreator` to manufacture a `Message` object.
- Three `convertAndSend()` methods accept an `Object` and automatically convert that `Object` into a `Message` behind the scenes.
- Three `convertAndSend()` methods automatically convert an `Object` to a `Message` but also accept a `MessagePostProcessor` to allow for customization of the `Message` before it's sent.

Moreover, each of these three method categories is composed of three overriding methods that are distinguished by how the JMS destination (queue or topic) is specified, as follows:

- One method accepts no destination parameter and sends the message to a default destination.
- One method accepts a `Destination` object that specifies the destination for the message.
- One method accepts a `String` that specifies the destination for the message by name.

Putting these methods to work, consider `JmsOrderMessagingService` in the next listing, which uses the most basic form of the `send()` method.

Listing 9.1 Sending an order with `.send()` to a default destination

```
package tacos.messaging;
import javax.jms.JMSException;
import javax.jms.Message;
import javax.jms.Session;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jms.core.JmsTemplate;
import org.springframework.jms.core.MessageCreator;
import org.springframework.stereotype.Service;

@Service
public class JmsOrderMessagingService implements OrderMessagingService {
    private JmsTemplate jms;

    @Autowired
    public JmsOrderMessagingService(JmsTemplate jms) {
        this.jms = jms;
    }

    @Override
    public void sendOrder(TacoOrder order) {
```

```

        jms.send(new MessageCreator() {
            @Override
            public Message createMessage(Session session)
                throws JMSException {
                return session.createObjectMessage(order);
            }
        });
    }
}

```

The `sendOrder()` method calls `jms.send()`, passing in an anonymous inner-class implementation of `MessageCreator`. That implementation overrides `createMessage()` to create a new object message from the given `TacoOrder` object.

Because the JMS-specific `JmsOrderMessagingService` implements the more generic `OrderMessagingService` interface, we can put this service to work by injecting it into the `OrderApiController` and calling `sendOrder()` when an order is created, as shown here:

```

@RestController
@RequestMapping(path="/api/orders",
    produces="application/json")
@CrossOrigin(origins="http://localhost:8080")
public class OrderApiController {

    private OrderRepository repo;
    private OrderMessagingService messageService;

    public OrderApiController(
        OrderRepository repo,
        OrderMessagingService messageService) {
        this.repo = repo;
        this.messageService = messageService;
    }

    @PostMapping(consumes="application/json")
    @ResponseStatus(HttpStatus.CREATED)
    public TacoOrder postOrder(@RequestBody TacoOrder order) {
        messageService.sendOrder(order);
        return repo.save(order);
    }

    ...
}

```

Now when you create an order through the Taco Cloud website, a message should be sent to the broker for routing to another application that will receive the order. We don't yet have anything to receive that message, though. Even so, you can use the Artemis console to view the contents of the queue. See the Artemis documentation at <http://mng.bz/aZx9> for details on how to do this.

I'm not sure about you, but I think the code in listing 9.1, although straightforward, is a bit clumsy. The ceremony involved in declaring an anonymous inner class complicates an otherwise simple method call. Recognizing that `MessageCreator` is a functional interface, you can tidy up the `sendOrder()` method a bit with a lambda, as shown next:

```
@Override
public void sendOrder(TacoOrder order) {
    jms.send(session -> session.createObjectMessage(order));
}
```

But notice that the call to `jms.send()` doesn't specify a destination. For this to work, you must also specify a default destination name with the `spring.jms.template.default-destination` property. For example, you could set the property in your `application.yml` file like this:

```
spring:
  jms:
    template:
      default-destination: tacocloud.order.queue
```

In many cases, using a default destination is the easiest choice. It lets you specify the destination name once, allowing the code to be concerned only with sending messages, without regard for where they're being sent. But if you ever need to send a message to a destination other than the default destination, you'll need to specify that destination as a parameter to `send()`.

One way of doing that is by passing a `Destination` object as the first parameter to `send()`. The easiest way to do this is to declare a `Destination` bean and then inject it into the bean that performs messaging. For example, the following bean declares the Taco Cloud order queue `Destination`:

```
@Bean
public Destination orderQueue() {
    return new ActiveMQQueue("tacocloud.order.queue");
}
```

This bean method can be added to any configuration class in the application that will be sending or receiving messages via JMS. For the sake of organization, it's best to add it to a configuration class designated for messaging configuration, such as `MessagingConfig`.

It's important to note that the `ActiveMQQueue` used here is actually from Artemis (from the `org.apache.activemq.artemis.jms.client` package). If you're using ActiveMQ (not Artemis), there's also a class named `ActiveMQQueue` (from the `org.apache.activemq.command` package).

If this `Destination` bean is injected into `JmsOrderMessagingService`, you can use it to specify the destination when calling `send()` as follows:

```

private Destination orderQueue;

@Autowired
public JmsOrderMessagingService(JmsTemplate jms,
                                Destination orderQueue) {
    this.jms = jms;
    this.orderQueue = orderQueue;
}

...

@Override
public void sendOrder(TacoOrder order) {
    jms.send(
        orderQueue,
        session -> session.createObjectMessage(order));
}

```

Specifying the destination with a `Destination` object like this affords you the opportunity to configure the `Destination` with more than just the destination name. But in practice, you'll almost never specify anything more than the destination name. It's often easier to just send the name as the first parameter to `send()`, as shown here:

```

@Override
public void sendOrder(TacoOrder order) {
    jms.send(
        "tacocloud.order.queue",
        session -> session.createObjectMessage(order));
}

```

Although the `send()` method isn't particularly difficult to use (especially when the `MessageCreator` is given as a lambda), a sliver of complexity is added by requiring that you provide a `MessageCreator`. Wouldn't it be simpler if you needed to specify only the object that's to be sent (and optionally the destination)? That describes succinctly how `convertAndSend()` works. Let's take a look.

CONVERTING MESSAGES BEFORE SENDING

The `JmsTemplates.convertAndSend()` method simplifies message publication by eliminating the need to provide a `MessageCreator`. Instead, you pass the object that's to be sent directly to `convertAndSend()`, and the object will be converted into a `Message` before being sent.

For example, the following reimplementaion of `sendOrder()` uses `convertAndSend()` to send a `TacoOrder` to a named destination:

```

@Override
public void sendOrder(TacoOrder order) {
    jms.convertAndSend("tacocloud.order.queue", order);
}

```

Just like the `send()` method, `convertAndSend()` will accept either a `Destination` or `String` value to specify the destination, or you can leave out the destination altogether to send the message to the default destination.

Whichever form of `convertAndSend()` you choose, the `TacoOrder` passed into `convertAndSend()` is converted into a `Message` before it's sent. Under the covers, this is achieved with an implementation of `MessageConverter` that does the dirty work of converting application domain objects to `Message` objects.

CONFIGURING A MESSAGE CONVERTER

`MessageConverter` is a Spring-defined interface that has only the following two methods to be implemented:

```
public interface MessageConverter {
    Message toMessage(Object object, Session session)
        throws JMSException, MessageConversionException;
    Object fromMessage(Message message)
}
```

Although this interface is simple enough to implement, you often won't need to create a custom implementation. Spring already offers a handful of implementations, such as those described in table 9.3.

Table 9.3 Spring message converters for common conversion tasks (all in the `org.springframework.jms.support.converter` package)

Message converter	What it does
<code>MappingJackson2MessageConverter</code>	Uses the Jackson 2 JSON library to convert messages to and from JSON
<code>MarshallingMessageConverter</code>	Uses JAXB to convert messages to and from XML
<code>MessagingMessageConverter</code>	Converts a <code>Message</code> from the messaging abstraction to and from a <code>Message</code> using an underlying <code>MessageConverter</code> for the payload and a <code>JmsHeaderMapper</code> to map the JMS headers to and from standard message headers
<code>SimpleMessageConverter</code>	Converts a <code>String</code> to and from a <code>TextMessage</code> , byte arrays to and from a <code>BytesMessage</code> , a <code>Map</code> to and from a <code>MapMessage</code> , and a <code>Serializable</code> to and from an <code>ObjectMessage</code>

`SimpleMessageConverter` is the default, but it requires that the object being sent implement `Serializable`. This may be a good idea, but you may prefer to use one of the other message converters, such as `MappingJackson2MessageConverter`, to avoid that restriction.

To apply a different message converter, all you must do is declare an instance of the chosen converter as a bean. For example, the following bean declaration will enable `MappingJackson2MessageConverter` to be used instead of `SimpleMessageConverter`:

```

@Bean
public MappingJackson2MessageConverter messageConverter() {
    MappingJackson2MessageConverter messageConverter =
        new MappingJackson2MessageConverter();
    messageConverter.setTypeIdPropertyName("_typeId");
    return messageConverter;
}

```

This bean method can be placed in any configuration class in the application that sends and receives messages with JMS, including alongside the Destination bean in `MessagingConfig`.

Notice that you called `setTypeIdPropertyName()` on the `MappingJackson2MessageConverter` before returning it. This is very important, because it enables the receiver to know what type to convert an incoming message to. By default, it will contain the fully qualified classname of the type being converted. But this method is somewhat inflexible, requiring that the receiver also have the same type, with the same fully qualified classname.

To allow for more flexibility, you can map a synthetic type name to the actual type by calling `setTypeIdMappings()` on the message converter. For example, the following change to the message converter bean method maps a synthetic `TacoOrder` type ID to the `TacoOrder` class:

```

@Bean
public MappingJackson2MessageConverter messageConverter() {
    MappingJackson2MessageConverter messageConverter =
        new MappingJackson2MessageConverter();
    messageConverter.setTypeIdPropertyName("_typeId");

    Map<String, Class<?>> typeIdMappings = new HashMap<String, Class<?>>();
    typeIdMappings.put("order", TacoOrder.class);
    messageConverter.setTypeIdMappings(typeIdMappings);

    return messageConverter;
}

```

Instead of the fully qualified classname being sent in the message's `_typeId` property, the value `TacoOrder` will be sent. In the receiving application, a similar message converter will have been configured, mapping `TacoOrder` to its own understanding of what an order is. That implementation of an order may be in a different package, have a different name, and even have a subset of the sender's `TacoOrder` properties.

POSTPROCESSING MESSAGES

Let's suppose that in addition to its lucrative web business, Taco Cloud has decided to open a few brick-and-mortar taco joints. Given that any of their restaurants could also be a fulfillment center for the web business, they need a way to communicate the source of an order to the kitchens at the restaurants. This will enable the kitchen staff to employ a different process for store orders than for web orders.

It would be reasonable to add a new source property to the `TacoOrder` object to carry this information, populating it with `WEB` for orders placed online and with `STORE` for orders placed in the stores. But that would require a change to both the website's `TacoOrder` class and the kitchen application's `TacoOrder` class when, in reality, it's information that's required only for the taco preparers.

An easier solution would be to add a custom header to the message to carry the order's source. If you were using the `send()` method to send the taco orders, this could easily be accomplished by calling `setStringProperty()` on the `Message` object as follows:

```
jms.send("tacocloud.order.queue",
    session -> {
        Message message = session.createObjectMessage(order);
        message.setStringProperty("X_ORDER_SOURCE", "WEB");
    });
```

The problem here is that you aren't using `send()`. By choosing to use `convertAndSend()`, the `Message` object is created under the covers, and you don't have access to it.

Fortunately, you have a way to tweak a `Message` created under the covers before it's sent. By passing in a `MessagePostProcessor` as the final parameter to `convertAndSend()`, you can do whatever you want with the `Message` after it has been created. The following code still uses `convertAndSend()`, but it also uses a `MessagePostProcessor` to add the `X_ORDER_SOURCE` header before the message is sent:

```
jms.convertAndSend("tacocloud.order.queue", order, new MessagePostProcessor()
{
    @Override
    public Message postProcessMessage(Message message) throws JMSEException {
        message.setStringProperty("X_ORDER_SOURCE", "WEB");
        return message;
    }
});
```

You may have noticed that `MessagePostProcessor` is a functional interface. This means that you can simplify it a bit by replacing the anonymous inner class with a lambda as shown here:

```
jms.convertAndSend("tacocloud.order.queue", order,
    message -> {
        message.setStringProperty("X_ORDER_SOURCE", "WEB");
        return message;
    });
```

Although you need this particular `MessagePostProcessor` for only this one call to `convertAndSend()`, you may find yourself using the same `MessagePostProcessor` for several different calls to `convertAndSend()`. In those cases, perhaps a method reference, shown next, is a better choice than a lambda, avoiding unnecessary code duplication:


```

@GetMapping("/convertAndSend/order")
public String convertAndSendOrder() {
    TacoOrder order = buildOrder();
    jms.convertAndSend("tacocloud.order.queue", order,
        this::addOrderSource);
    return "Convert and sent order";
}

private Message addOrderSource(Message message) throws JMSException {
    message.setStringProperty("X_ORDER_SOURCE", "WEB");
    return message;
}

```

You've now seen several ways of sending messages. But it does no good to send a message if nobody ever receives it. Let's look at how you can receive messages with Spring JMS.

9.1.3 *Receiving JMS messages*

When it comes to consuming messages, you have the choice of a *pull model*, where your code requests a message and waits until one arrives, or a *push model*, in which messages are handed to your code as they become available.

`JmsTemplate` offers several methods for receiving messages, but all of them use a pull model. You call one of those methods to request a message, and the thread is blocked until a message is available (which could be immediately or it might take a while).

On the other hand, you also have the option of using a push model, wherein you define a message listener that's invoked any time a message is available.

Both options are suitable for a variety of use cases. It's generally accepted that the push model is the best choice, because it doesn't block a thread. But in some use cases, a listener could be overburdened if messages arrive too quickly. The pull model enables a consumer to declare that they're ready to process a new message.

Let's look at both ways of receiving messages. We'll start with the pull model offered by `JmsTemplate`.

RECEIVING WITH `JmsTemplate`

`JmsTemplate` offers several methods for pulling methods from the broker, including the following:

```

Message receive() throws JmsException;
Message receive(Destination destination) throws JmsException;
Message receive(String destinationName) throws JmsException;

Object receiveAndConvert() throws JmsException;
Object receiveAndConvert(Destination destination) throws JmsException;
Object receiveAndConvert(String destinationName) throws JmsException;

```

As you can see, these six methods mirror the `send()` and `convertAndSend()` methods from `JmsTemplate`. The `receive()` methods receive a raw `Message`, whereas the

`receiveAndConvert()` methods use a configured message converter to convert messages into domain types. And for each of these, you can specify either a `Destination` or a `String` containing the destination name, or you can pull a message from the default destination.

To see these in action, you'll write some code that pulls an `TacoOrder` from the `tacocloud.order.queue` destination. The following listing shows `OrderReceiver`, a service component that receives order data using `JmsTemplate.receive()`.

Listing 9.2 Pulling orders from a queue

```
package tacos.kitchen.messaging.jms;
import javax.jms.Message;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jms.core.JmsTemplate;
import org.springframework.jms.support.converter.MessageConverter;
import org.springframework.stereotype.Component;

@Component
public class JmsOrderReceiver implements OrderReceiver {
    private JmsTemplate jms;
    private MessageConverter converter;

    @Autowired
    public JmsOrderReceiver(JmsTemplate jms, MessageConverter converter) {
        this.jms = jms;
        this.converter = converter;
    }

    public TacoOrder receiveOrder() {
        Message message = jms.receive("tacocloud.order.queue");
        return (TacoOrder) converter.fromMessage(message);
    }
}
```

Here you've used a `String` to specify the destination from which to pull an order. The `receive()` method returns an unconverted `Message`. But what you really need is the `TacoOrder` that's inside of the `Message`, so the very next thing that happens is that you use an injected message converter to convert the message. The type ID property in the message will guide the converter in converting it to a `TacoOrder`, but it's returned as an `Object` that requires casting before you can return it.

Receiving a raw `Message` object might be useful in some cases where you need to inspect the message's properties and headers. But often you need only the payload. Converting that payload to a domain type is a two-step process and requires that the message converter be injected into the component. When you care only about the message's payload, `receiveAndConvert()` is a lot simpler. The next listing shows how `JmsOrderReceiver` could be reworked to use `receiveAndConvert()` instead of `receive()`.

Listing 9.3 Receiving an already-converted `TacoOrder` object

```

package tacos.kitchen.messaging.jms;

import org.springframework.jms.core.JmsTemplate;
import org.springframework.stereotype.Component;
import tacos.TacoOrder;
import tacos.kitchen.OrderReceiver;

@Component
public class JmsOrderReceiver implements OrderReceiver {

    private JmsTemplate jms;

    public JmsOrderReceiver(JmsTemplate jms) {
        this.jms = jms;
    }

    @Override
    public TacoOrder receiveOrder() {
        return (TacoOrder) jms.receiveAndConvert("tacocloud.order.queue");
    }
}

```

This new version of `JmsOrderReceiver` has a `receiveOrder()` method that has been reduced to only one line. Plus, you no longer need to inject a `MessageConverter`, because all of the message conversion will be done behind the scenes in `receiveAndConvert()`.

Before moving on, let's consider how `receiveOrder()` might be used in the Taco Cloud kitchen application. A food preparer at one of Taco Cloud's kitchens might push a button or take some action to indicate that they're ready to start building tacos. At that point, `receiveOrder()` would be invoked and the call to `receive()` or `receiveAndConvert()` would block. Nothing else would happen until an order message is ready. Once an order arrives, it will be returned from `receiveOrder()`, and its information will be used to display the details of the order for the food preparer to get to work. This seems like a natural choice for a pull model.

Now let's see how a push model works by declaring a JMS listener.

DECLARING MESSAGE LISTENERS

Unlike the pull model, where an explicit call to `receive()` or `receiveAndConvert()` was required to receive a message, a message listener is a passive component that's idle until a message arrives.

To create a message listener that reacts to JMS messages, you simply annotate a method in a component with `@JmsListener`. The next listing shows a new `OrderListener` component that listens passively for messages, rather than actively requesting them.

Listing 9.4 An OrderListener component that listens for orders

```
package tacos.kitchen.messaging.jms.listener;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Profile;
import org.springframework.jms.annotation.JmsListener;
import org.springframework.stereotype.Component;

import tacos.TacoOrder;
import tacos.kitchen.KitchenUI;

@Profile("jms-listener")
@Component
public class OrderListener {

    private KitchenUI ui;

    @Autowired
    public OrderListener(KitchenUI ui) {
        this.ui = ui;
    }

    @JmsListener(destination = "tacocloud.order.queue")
    public void receiveOrder(TacoOrder order) {
        ui.displayOrder(order);
    }

}
```

The `receiveOrder()` method is annotated with `JmsListener` to “listen” for messages on the `tacocloud.order.queue` destination. It doesn’t deal with `JmsTemplate`, nor is it explicitly invoked by your application code. Instead, framework code within Spring waits for messages to arrive on the specified destination, and when they arrive, the `receiveOrder()` method is invoked automatically with the message’s `TacoOrder` payload as a parameter.

In many ways, the `@JmsListener` annotation is like one of Spring MVC’s request mapping annotations, such as `@GetMapping` or `@PostMapping`. In Spring MVC, methods annotated with one of the request mapping methods react to requests to a specified path. Similarly, methods that are annotated with `@JmsListener` react to messages that arrive in a destination.

Message listeners are often touted as the best choice because they don’t block and are able to handle multiple messages quickly. In the context of the Taco Cloud application, however, perhaps they aren’t the best choice. The food preparers are a significant bottleneck in the system and may not be able to prepare tacos as quickly as orders come in. A food preparer may have half-fulfilled an order when a new order is displayed on the screen. The kitchen user interface would need to buffer the orders as they arrive to avoid overburdening the kitchen staff.

That's not to say that message listeners are bad. On the contrary, they're a perfect fit when messages can be handled quickly. But when the message handlers need to be able to ask for more messages on their own timing, the pull model offered by `JmsTemplate` seems more fitting.

Because JMS is defined by a standard Java specification and supported by many message broker implementations, it's a common choice for messaging in Java. But JMS has a few shortcomings, not the least of which is that as a Java specification, its use is limited to Java applications. Newer messaging options such as RabbitMQ and Kafka address these shortcomings and are available for other languages and platforms beyond the JVM. Let's set JMS aside and see how you could have implemented your taco order messaging with RabbitMQ.

9.2 *Working with RabbitMQ and AMQP*

As arguably the most prominent implementation of AMQP, RabbitMQ offers a more advanced message-routing strategy than JMS. Whereas JMS messages are addressed with the name of a destination from which the receiver will retrieve them, AMQP messages are addressed with the name of an exchange and a routing key, which are decoupled from the queue to which the receiver is listening. This relationship between an exchange and queues is illustrated in figure 9.2.

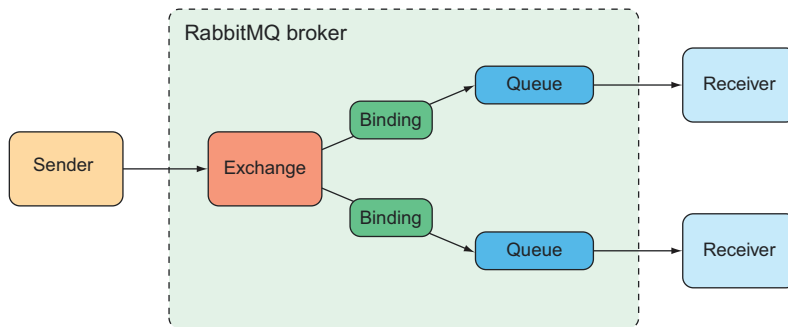


Figure 9.2 Messages sent to a RabbitMQ exchange are routed to one or more queues, based on routing keys and bindings.

When a message arrives at the RabbitMQ broker, it goes to the exchange for which it was addressed. The exchange is responsible for routing it to one or more queues, depending on the type of exchange, the binding between the exchange and queues, and the value of the message's routing key.

There are several different kinds of exchanges, including the following:

- *Default*—A special exchange that's automatically created by the broker. It routes messages to queues whose name is the same as the message's routing key. All queues will automatically be bound to the default exchange.

- *Direct*—Routes messages to a queue whose binding key is the same as the message’s routing key.
- *Topic*—Routes a message to one or more queues where the binding key (which may contain wildcards) matches the message’s routing key.
- *Fanout*—Routes messages to all bound queues without regard for binding keys or routing keys.
- *Headers*—Similar to a topic exchange, except that routing is based on message header values rather than routing keys.
- *Dead letter*—A catchall for any messages that are undeliverable (meaning they don’t match any defined exchange-to-queue binding).

The simplest forms of exchanges are default and fanout—these roughly correspond to a JMS queue and topic. But the other exchanges allow you to define more flexible routing schemes.

The most important thing to understand is that messages are sent to exchanges with routing keys and they’re consumed from queues. How they get from an exchange to a queue depends on the binding definitions and what best suits your use cases.

Which exchange type you use and how you define the bindings from exchanges to queues has little bearing on how messages are sent and received in your Spring applications. Therefore, we’ll focus on how to write code that sends and receives messages with Rabbit.

NOTE For a more detailed discussion on how best to bind queues to exchanges, see *RabbitMQ in Depth* by Gavin Roy (Manning, 2017) or *RabbitMQ in Action* by Alvaro Videla and Jason J. W. Williams (Manning, 2012).

9.2.1 Adding RabbitMQ to Spring

Before you can start sending and receiving RabbitMQ messages with Spring, you’ll need to add Spring Boot’s AMQP starter dependency to your build in place of the Artemis or ActiveMQ starter you added in the previous section, as shown here:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

Adding the AMQP starter to your build will trigger autoconfiguration that will create an AMQP connection factory and `RabbitTemplate` beans, as well as other supporting components. Simply adding this dependency is all you need to do to start sending and receiving messages from a RabbitMQ broker with Spring. But there are a handful of useful properties you’ll want to know about, listed in table 9.4.

For development purposes, you’ll probably have a RabbitMQ broker that doesn’t require authentication running on your local machine, listening on port 5672. These properties likely won’t get much use while you’re still in development, but they’ll no doubt prove useful when your applications move into production.

Table 9.4 Properties for configuring the location and credentials of a RabbitMQ broker

Property	Description
<code>spring.rabbitmq.addresses</code>	A comma-separated list of RabbitMQ broker addresses
<code>spring.rabbitmq.host</code>	The broker's host (defaults to localhost)
<code>spring.rabbitmq.port</code>	The broker's port (defaults to 5672)
<code>spring.rabbitmq.username</code>	The username for accessing the broker (optional)
<code>spring.rabbitmq.password</code>	The password for accessing the broker (optional)

RUNNING A RABBITMQ BROKER If you don't already have a RabbitMQ broker to work with, you have several options for running RabbitMQ on your local machine. See the official RabbitMQ documentation at <https://www.rabbitmq.com/download.html> for the latest instructions for running RabbitMQ.

For example, suppose that as you move into production, your RabbitMQ broker is on a server named `rabbit.tacocloud.com`, listening on port 5673, and requiring credentials. In that case, the following configuration in your `application.yml` file will set those properties when the `prod` profile is active:

```
spring:
  profiles: prod
  rabbitmq:
    host: rabbit.tacocloud.com
    port: 5673
    username: tacoweb
    password: l3tm31n
```

Now that RabbitMQ is configured in your application, it's time to start sending messages with `RabbitTemplate`.

9.2.2 *Sending messages with RabbitTemplate*

At the core of Spring's support for RabbitMQ messaging is `RabbitTemplate`. `RabbitTemplate` is similar to `JmsTemplate` and offers a similar set of methods. As you'll see, however, some subtle differences align with the unique way that RabbitMQ works.

With regard to sending messages with `RabbitTemplate`, the `send()` and `convertAndSend()` methods parallel the same-named methods from `JmsTemplate`. But unlike the `JmsTemplate` methods, which route messages only to a given queue or topic, `RabbitTemplate` methods send messages in terms of exchanges and routing keys. Here are a few of the most relevant methods for sending messages with `RabbitTemplate`:¹

```
// Send raw messages
void send(Message message) throws AmqpException;
void send(String routingKey, Message message) throws AmqpException;
```

¹ These methods are defined by `AmqpTemplate`, an interface implemented by `RabbitTemplate`.

```

void send(String exchange, String routingKey, Message message)
    throws AmqpException;

// Send messages converted from objects
void convertAndSend(Object message) throws AmqpException;
void convertAndSend(String routingKey, Object message)
    throws AmqpException;
void convertAndSend(String exchange, String routingKey,
    Object message) throws AmqpException;

// Send messages converted from objects with post-processing
void convertAndSend(Object message, MessagePostProcessor mPP)
    throws AmqpException;
void convertAndSend(String routingKey, Object message,
    MessagePostProcessor messagePostProcessor)
    throws AmqpException;
void convertAndSend(String exchange, String routingKey,
    Object message,
    MessagePostProcessor messagePostProcessor)
    throws AmqpException;

```

As you can see, these methods follow a pattern similar to their twins in `JmsTemplate`. The first three `send()` methods all send a raw `Message` object. The next three `convertAndSend()` methods accept an object that will be converted to a `Message` behind the scenes before being sent. The final three `convertAndSend()` methods are like the previous three, but they accept a `MessagePostProcessor` that can be used to manipulate the `Message` object before it's sent to the broker.

These methods differ from their `JmsTemplate` counterparts in that they accept `String` values to specify an exchange and routing key, rather than a destination name (or `Destination` object). The methods that don't take an exchange will have their messages sent to the default exchange. Likewise, the methods that don't take a routing key will have their messages routed with a default routing key.

Let's put `RabbitTemplate` to work sending taco orders. One way you can do that is by using the `send()` method, as shown in listing 9.5. But before you can call `send()`, you'll need to convert a `TacoOrder` object to a `Message`. That could be a tedious job, if not for the fact that `RabbitTemplate` makes its message converter readily available with a `getMessageConverter()` method.

Listing 9.5 Sending a message with `RabbitTemplate.send()`

```

package tacos.messaging;
import org.springframework.amqp.core.Message;
import org.springframework.amqp.core.MessageProperties;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import
    org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import tacos.Order;

```



```

@Service
public class RabbitOrderMessagingService
    implements OrderMessagingService {
    private RabbitTemplate rabbit;

    @Autowired
    public RabbitOrderMessagingService(RabbitTemplate rabbit) {
        this.rabbit = rabbit;
    }

    public void sendOrder(TacoOrder order) {
        MessageConverter converter = rabbit.getMessageConverter();
        MessageProperties props = new MessageProperties();
        Message message = converter.toMessage(order, props);
        rabbit.send("tacocloud.order", message);
    }
}

```

You'll notice that `RabbitOrderMessagingService` implements `OrderMessagingService`, just like `JmsOrderMessagingService`. This means that it can be injected into `OrderApiController` the same way to send order messages when an order is placed. Because we don't yet have anything to receive those messages, though, you can use the RabbitMQ browser-based management console. See <https://www.rabbitmq.com/management.html> for details on how to enable and use the RabbitMQ console.

Once you have a `MessageConverter` in hand, it's simple work to convert a `TacoOrder` to a `Message`. You must supply any message properties with a `MessageProperties`, but if you don't need to set any such properties, a default instance of `MessageProperties` is fine. Then, all that's left is to call `send()`, passing in the exchange and routing key (both of which are optional) along with the message. In this example, you're specifying only the routing key—`tacocloud.order`—along with the message, so the default exchange will be used.

Speaking of default exchanges, the default exchange name is `"` (an empty `String`), which corresponds to the default exchange that's automatically created by the RabbitMQ broker. Likewise, the default routing key is `"` (whose routing is dependent upon the exchange and bindings in question). You can override these defaults by setting the `spring.rabbitmq.template.exchange` and `spring.rabbitmq.template.routing-key` properties as follows:

```

spring:
  rabbitmq:
    template:
      exchange: tacocloud.order
      routing-key: kitchens.central

```

In this case, all messages sent without specifying an exchange will automatically be sent to the exchange whose name is `tacocloud.order`. If the routing key is also unspecified in the call to `send()` or `convertAndSend()`, the messages will have a routing key of `kitchens.central`.

Creating a `Message` object from the message converter is easy enough, but it's even easier to use `convertAndSend()` to let `RabbitTemplate` handle all of the conversion work for you, as shown next:

```
public void sendOrder(TacoOrder order) {
    rabbit.convertAndSend("tacocloud.order", order);
}
```

CONFIGURING A MESSAGE CONVERTER

By default, message conversion is performed with `SimpleMessageConverter`, which is able to convert simple types (like `String`) and `Serializable` objects to `Message` objects. But Spring offers several message converters for `RabbitTemplate`, including the following:

- *Jackson2JsonMessageConverter*—Converts objects to and from JSON using the Jackson 2 JSON processor
- *MarshallingMessageConverter*—Converts using a Spring Marshaller and Unmarshaller
- *SerializerMessageConverter*—Converts `String` and native objects of any kind using Spring's `Serializer` and `Deserializer` abstractions
- *SimpleMessageConverter*—Converts `String`, byte arrays, and `Serializable` types
- *ContentTypeDelegatingMessageConverter*—Delegates to another `MessageConverter` based on the `contentType` header
- *MessagingMessageConverter*—Delegates to an underlying `MessageConverter` for the message conversion and to an `AmqpHeaderConverter` for the headers

If you need to change the message converter, just configure a bean of type `MessageConverter`. For example, for JSON-based message conversion, you can configure a `Jackson2JsonMessageConverter` like this:

```
@Bean
public Jackson2JsonMessageConverter messageConverter() {
    return new Jackson2JsonMessageConverter();
}
```

Spring Boot autoconfiguration will discover this bean and inject it into `RabbitTemplate` in place of the default message converter.

SETTING MESSAGE PROPERTIES

As with JMS, you may need to set some headers in the messages you send. For example, let's say you need to send an `X_ORDER_SOURCE` for all orders submitted through the Taco Cloud website. When creating your own `Message` objects, you can set the header through the `MessageProperties` instance you give to the message converter. Revisiting the `sendOrder()` method from listing 9.5, you only need one additional line of code to set the header, as shown next:

```
public void sendOrder(TacoOrder order) {
    MessageConverter converter = rabbit.getMessageConverter();
    MessageProperties props = new MessageProperties();
    props.setHeader("X_ORDER_SOURCE", "WEB");
    Message message = converter.toMessage(order, props);
    rabbit.send("tacocloud.order", message);
}
```

When using `convertAndSend()`, however, you don't have quick access to the `MessageProperties` object. A `MessagePostProcessor` can help you with that, though, as shown here:

```
public void sendOrder(TacoOrder order) {
    rabbit.convertAndSend("tacocloud.order.queue", order,
        new MessagePostProcessor() {
            @Override
            public Message postProcessMessage(Message message)
                throws AmqpException {
                MessageProperties props = message.getMessageProperties();
                props.setHeader("X_ORDER_SOURCE", "WEB");
                return message;
            }
        });
}
```

Here you supply `convertAndSend()` with an anonymous inner-class implementation of `MessagePostProcessor`. In the `postProcessMessage()` method, you pull the `MessageProperties` from the `Message` and then call `setHeader()` to set the `X_ORDER_SOURCE` header.

Now that you've seen how to send messages with `RabbitTemplate`, let's switch our focus over to the code that receives messages from a `RabbitMQ` queue.

9.2.3 *Receiving messages from RabbitMQ*

You've seen that sending messages with `RabbitTemplate` doesn't differ much from sending messages with `JmsTemplate`. And as it turns out, receiving messages from a `RabbitMQ` queue isn't very different than from `JMS`.

As with `JMS`, you have the following two choices:

- Pulling messages from a queue with `RabbitTemplate`
- Having messages pushed to a `@RabbitListener`-annotated method

Let's start by looking at the pull-based `RabbitTemplate.receive()` method.

RECEIVING MESSAGES WITH `RABBITTEMPLATE`

`RabbitTemplate` comes with several methods for pulling messages from a queue. A few of the most useful ones are listed here:

```
// Receive messages
Message receive() throws AmqpException;
Message receive(String queueName) throws AmqpException;
```

```

Message receive(long timeoutMillis) throws AmqpException;
Message receive(String queueName, long timeoutMillis) throws AmqpException;

// Receive objects converted from messages
Object receiveAndConvert() throws AmqpException;
Object receiveAndConvert(String queueName) throws AmqpException;
Object receiveAndConvert(long timeoutMillis) throws AmqpException;
Object receiveAndConvert(String queueName, long timeoutMillis)
    throws AmqpException;

// Receive type-safe objects converted from messages
<T> T receiveAndConvert(ParameterizedTypeReference<T> type)
    throws AmqpException;
<T> T receiveAndConvert(
    String queueName, ParameterizedTypeReference<T> type)
    throws AmqpException;
<T> T receiveAndConvert(
    long timeoutMillis, ParameterizedTypeReference<T> type)
    throws AmqpException;
<T> T receiveAndConvert(String queueName, long timeoutMillis,
    ParameterizedTypeReference<T> type)
    throws AmqpException;

```

These methods are the mirror images of the `send()` and `convertAndSend()` methods described earlier. Whereas `send()` is used to send raw `Message` objects, `receive()` receives raw `Message` objects from a queue. Likewise, `receiveAndConvert()` receives messages and uses a message converter to convert them into domain objects before returning them.

But a few obvious differences occur in the method signatures. First, none of these methods take an exchange or routing key as a parameter. That's because exchanges and routing keys are used to route messages to queues, but once the messages are in the queue, their next destination is the consumer who pulls them off the queue. Consuming applications needn't concern themselves with exchanges or routing keys. A queue is the only thing the consuming applications need to know about.

You'll also notice that many of the methods accept a long parameter to indicate a time-out for receiving the messages. By default, the receive time-out is 0 milliseconds. That is, a call to `receive()` will return immediately, potentially with a null value if no messages are available. This is a marked difference from how the `receive()` methods behave in `JmsTemplate`. By passing in a time-out value, you can have the `receive()` and `receiveAndConvert()` methods block until a message arrives or until the time-out expires. But even with a non-zero time-out, your code will need to be ready to deal with a null return.

Let's see how you can put this in action. The next listing shows a new Rabbit-based implementation of `OrderReceiver` that uses `RabbitTemplate` to receive orders.

Listing 9.6 Pulling orders from RabbitMQ with `RabbitTemplate`

```

package tacos.kitchen.messaging.rabbit;
import org.springframework.amqp.core.Message;
import org.springframework.amqp.rabbit.core.RabbitTemplate;

```

```

import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class RabbitOrderReceiver {
    private RabbitTemplate rabbit;
    private MessageConverter converter;

    @Autowired
    public RabbitOrderReceiver(RabbitTemplate rabbit) {
        this.rabbit = rabbit;
        this.converter = rabbit.getMessageConverter();
    }

    public TacoOrder receiveOrder() {
        Message message = rabbit.receive("tacocloud.order");
        return message != null
            ? (TacoOrder) converter.fromMessage(message)
            : null;
    }
}

```

The `receiveOrder()` method is where all of the action takes place. It makes a call to the `receive()` method on the injected `RabbitTemplate` to pull an order from the `tacocloud.order` queue. It provides no time-out value, so you can assume only that the call returns immediately with either a `Message` or `null`. If a `Message` is returned, you use the `MessageConverter` from the `RabbitTemplate` to convert the `Message` to a `TacoOrder`. On the other hand, if `receive()` returns `null`, you'll return a `null`.

Depending on the use case, you may be able to tolerate a small delay. In the Taco Cloud kitchen's overhead display, for example, you can possibly wait a while if no orders are available. Let's say you decide to wait up to 30 seconds before giving up. Then the `receiveOrder()` method can be changed to pass a 30,000 millisecond delay to `receive()` as follows:

```

public TacoOrder receiveOrder() {
    Message message = rabbit.receive("tacocloud.order.queue", 30000);
    return message != null
        ? (TacoOrder) converter.fromMessage(message)
        : null;
}

```

If you're like me, seeing a hardcoded number like that gives you a bit of discomfort. You might be thinking that it'd be a good idea to create a `@ConfigurationProperties`-annotated class so you could configure that time-out with a Spring Boot configuration property. I'd agree with you, if it weren't for the fact that Spring Boot already offers such a configuration property. If you want to set the time-out via configuration, simply remove the time-out value in the call to `receive()` and set it in your configuration with the `spring.rabbitmq.template.receive-timeout` property like so:

```
spring:
  rabbitmq:
    template:
      receive-timeout: 30000
```

Back in the `receiveOrder()` method, notice that you had to use the message converter from `RabbitTemplate` to convert the incoming `Message` object to a `TacoOrder` object. But if the `RabbitTemplate` is carrying a message converter around, why can't it do the conversion for you? That's precisely what the `receiveAndConvert()` method is for. Using `receiveAndConvert()`, you can rewrite `receiveOrder()` like this:

```
public TacoOrder receiveOrder() {
    return (TacoOrder) rabbit.receiveAndConvert("tacocloud.order.queue");
}
```

That's a lot simpler, isn't it? The only troubling thing I see is the cast from `Object` to `TacoOrder`. There's an alternative to casting, though. Instead, you can pass a `ParameterizedTypeReference` to `receiveAndConvert()` to receive a `TacoOrder` object directly as follows:

```
public TacoOrder receiveOrder() {
    return rabbit.receiveAndConvert("tacocloud.order.queue",
        new ParameterizedTypeReference<Order>() {});
}
```

It's debatable whether that's better than casting, but it is a more type-safe approach than casting. The only requirement to using a `ParameterizedTypeReference` with `receiveAndConvert()` is that the message converter must be an implementation of `SmartMessageConverter`; `Jackson2JsonMessageConverter` is the only out-of-the-box implementation to choose from.

The pull model offered by `JmsTemplate` fits a lot of use cases, but often it's better to have code that listens for messages and that's invoked when messages arrive. Let's see how you can write message-driven beans that respond to RabbitMQ messages.

HANDLING RABBITMQ MESSAGES WITH LISTENERS

For message-driven RabbitMQ beans, Spring offers `RabbitListener`, the RabbitMQ counterpart to `JmsListener`. To specify that a method should be invoked when a message arrives in a RabbitMQ queue, annotate a bean's method with `@RabbitListener`.

For example, the following listing shows a RabbitMQ implementation of `OrderReceiver` that's annotated to listen for order messages rather than to poll for them with `RabbitTemplate`.

Listing 9.7 Declaring a method as a RabbitMQ message listener

```
package tacos.kitchen.messaging.rabbit.listener;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
```

```

import tacos.TacoOrder;
import tacos.kitchen.KitchenUI;

@Component
public class OrderListener {

    private KitchenUI ui;

    @Autowired
    public OrderListener(KitchenUI ui) {
        this.ui = ui;
    }

    @RabbitListener(queues = "tacocloud.order.queue")
    public void receiveOrder(TacoOrder order) {
        ui.displayOrder(order);
    }
}

```

You'll no doubt notice that this looks remarkably like the code from listing 9.4. Indeed, the only thing that changed was the listener annotation—from `@JmsListener` to `@RabbitListener`. As wonderful as `@RabbitListener` is, this near duplication of code leaves me with little to say about `@RabbitListener` that I haven't already said about `@JmsListener`. They're both great for writing code that responds to messages that are pushed to them from their respective brokers—a JMS broker for `@JmsListener` and a RabbitMQ broker for `@RabbitListener`.

Although you may sense a lack of enthusiasm about `@RabbitListener` in that previous paragraph, be certain that isn't my intent. In truth, the fact that `@RabbitListener` works much like `@JmsListener` is actually quite exciting! It means you don't need to learn a completely different programming model when working with RabbitMQ vs. Artemis or ActiveMQ. The same excitement holds true for the similarities between `RabbitTemplate` and `JmsTemplate`.

Let's hold on to that excitement as we wrap up this chapter by looking at one more messaging option supported by Spring: Apache Kafka.

9.3 *Messaging with Kafka*

Apache is the newest messaging option we're examining in this chapter. At a glance, Kafka is a message broker just like ActiveMQ, Artemis, or Rabbit. But Kafka has a few unique tricks up its sleeves.

Kafka is designed to run in a cluster, affording great scalability. And by partitioning its topics across all instances in the cluster, it's very resilient. Whereas RabbitMQ deals primarily with queues in exchanges, Kafka utilizes topics only to offer pub/sub messaging.

Kafka topics are replicated across all brokers in the cluster. Each node in the cluster acts as a leader for one or more topics, being responsible for that topic's data and replicating it to the other nodes in the cluster.

Going a step further, each topic can be split into multiple partitions. In that case, each node in the cluster is the leader for one or more partitions of a topic, but not for the entire topic. Responsibility for the topic is split across all nodes. Figure 9.3 illustrates how this works.

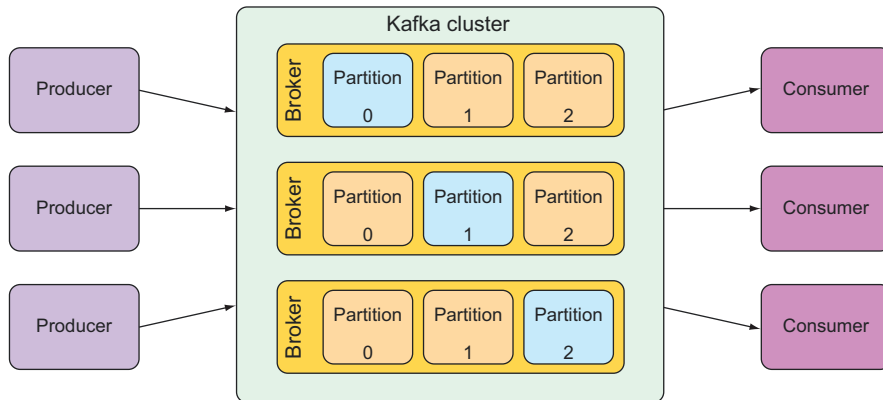


Figure 9.3 A Kafka cluster is composed of multiple brokers, each acting as a leader for partitions of the topics.

Due to Kafka’s unique architecture, I encourage you to read more about it in *Kafka in Action* by Dylan Scott, Viktor Gamov, and Dave Klein (Manning, 2021). For our purposes, we’ll focus on how to send messages to and receive them from Kafka with Spring.

9.3.1 Setting up Spring for Kafka messaging

To start using Kafka for messaging, you’ll need to add the appropriate dependencies to your build. Unlike the JMS and RabbitMQ options, however, there isn’t a Spring Boot starter for Kafka. Have no fear, though; you’ll only need one dependency, shown next:

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

This one dependency brings everything you need for Kafka to the project. What’s more, its presence will trigger Spring Boot autoconfiguration for Kafka that will, among other things, arrange for a `KafkaTemplate` in the Spring application context. All you need to do is inject the `KafkaTemplate` and go to work sending and receiving messages.

Before you start sending and receiving messages, however, you should be aware of a few properties that will come in handy when working with Kafka. Specifically, `KafkaTemplate` defaults to work with a Kafka broker on localhost, listening on port 9092.


```

ListenableFuture<SendResult<K, V>> send(String topic,
    Integer partition, Long timestamp, K key, V data);
ListenableFuture<SendResult<K, V>> send(ProducerRecord<K, V> record);
ListenableFuture<SendResult<K, V>> send(Message<?> message);

ListenableFuture<SendResult<K, V>> sendDefault(V data);
ListenableFuture<SendResult<K, V>> sendDefault(K key, V data);
ListenableFuture<SendResult<K, V>> sendDefault(Integer partition,
    K key, V data);
ListenableFuture<SendResult<K, V>> sendDefault(Integer partition,
    Long timestamp, K key, V data);

```

The first thing you may have noticed is that there are no `convertAndSend()` methods. That's because `KafkaTemplate` is typed with generics and is able to deal with domain types directly when sending messages. In a way, all of the `send()` methods are doing the job of `convertAndSend()`.

You may also have noticed that there are several parameters to `send()` and `sendDefault()` that are quite different from what you used with JMS and Rabbit. When sending messages in Kafka, you can specify the following parameters to guide how the message is sent as follows:

- The topic to which to send the message (required for `send()`)
- A partition to which to write the topic (optional)
- A key to send on the record (optional)
- A timestamp (optional; defaults to `System.currentTimeMillis()`)
- The payload (required)

The topic and payload are the two most important parameters. Partitions and keys have little effect on how you use `KafkaTemplate`, aside from being extra information provided as parameters to `send()` and `sendDefault()`. For our purposes, we're going to focus on sending the message payload to a given topic and not worry ourselves with partitions and keys.

For the `send()` method, you can also choose to send a `ProducerRecord`, which is little more than a type that captures all of the preceding parameters in a single object. You can also send a `Message` object, but doing so would require you to convert your domain objects into a `Message`. Generally, it's easier to use one of the other methods rather than to create and send a `ProducerRecord` or `Message` object.

Using the `KafkaTemplate` and its `send()` method, you can write a Kafka-based implementation of `OrderMessagingService`. The following listing shows what such an implementation might look like.

Listing 9.8 Sending orders with `KafkaTemplate`

```

package tacos.messaging;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;
import tacos.TacoOrder;

```

```

@Service
public class KafkaOrderMessagingService
    implements OrderMessagingService {

    private KafkaTemplate<String, TacoOrder> kafkaTemplate;

    @Autowired
    public KafkaOrderMessagingService(
        KafkaTemplate<String, TacoOrder> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    @Override
    public void sendOrder(TacoOrder order) {
        kafkaTemplate.send("tacocloud.orders.topic", order);
    }
}

```

In this new implementation of `OrderMessagingService`, the `sendOrder()` method uses the `send()` method of the injected `KafkaTemplate` to send a `TacoOrder` to the topic named `tacocloud.orders.topic`. Except for the word “Kafka” scattered throughout the code, this isn’t much different than the code you wrote for JMS and Rabbit. And, just like those other implementations of `OrderMessagingService`, it can be injected into `OrderApiController` and used to send orders through Kafka when orders are placed via the `/api/orders` endpoint.

Until we create a Kafka implementation of the message receiver, you’ll need a console to view what was sent. There are several management consoles available for Kafka, including `Offset Explorer` (<https://www.kafkatool.com/>) and Confluent’s `Apache Kafka UI` (<http://mng.bz/g1P8>).

If you set a default topic, you can simplify the `sendOrder()` method slightly. First, set your default topic to `tacocloud.orders.topic` by setting the `spring.kafka.template.default-topic` property as follows:

```

spring:
  kafka:
    bootstrap-servers:
      - localhost:9092
    template:
      default-topic: tacocloud.orders.topic

```

Then, in the `sendOrder()` method, you can call `sendDefault()` instead of `send()` and not specify the topic name, as shown here:

```

@Override
public void sendOrder(TacoOrder order) {
    kafkaTemplate.sendDefault(order);
}

```

Now that your message-sending code has been written, let’s turn our attention to writing code that will receive those messages from Kafka.

9.3.3 Writing Kafka listeners

Aside from the unique method signatures for `send()` and `sendDefault()`, `KafkaTemplate` differs from `JmsTemplate` and `RabbitTemplate` in that it doesn't offer any methods for receiving messages. That means the only way to consume messages from a Kafka topic using Spring is to write a message listener.

For Kafka, message listeners are defined as methods that are annotated with `@KafkaListener`. The `@KafkaListener` annotation is roughly analogous to `@JmsListener` and `@RabbitListener` and is used in much the same way. The next listing shows what your listener-based order receiver might look like if written for Kafka.

Listing 9.9 Receiving orders with `@KafkaListener`

```
package tacos.kitchen.messaging.kafka.listener;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import tacos.Order;
import tacos.kitchen.KitchenUI;

@Component
public class OrderListener {

    private KitchenUI ui;

    @Autowired
    public OrderListener(KitchenUI ui) {
        this.ui = ui;
    }

    @KafkaListener(topics="tacocloud.orders.topic")
    public void handle(TacoOrder order) {
        ui.displayOrder(order);
    }

}
```

The `handle()` method is annotated with `@KafkaListener` to indicate that it should be invoked when a message arrives in the topic named `tacocloud.orders.topic`. As it's written in listing 9.9, only a `TacoOrder` (the payload) is given to `handle()`. But if you need additional metadata from the message, it can also accept a `ConsumerRecord` or `Message` object.

For example, the following implementation of `handle()` accepts a `ConsumerRecord` so that you can log the partition and timestamp of the message:

```
@KafkaListener(topics="tacocloud.orders.topic")
public void handle(
    TacoOrder order, ConsumerRecord<String, TacoOrder> record) {
    log.info("Received from partition {} with timestamp {}",
        record.partition(), record.timestamp());
}
```

```
    ui.displayOrder(order);  
}
```

Similarly, you could ask for a `Message` instead of a `ConsumerRecord` and achieve the same thing, as shown here:

```
@KafkaListener(topics="tacocloud.orders.topic")  
public void handle(Order order, Message<Order> message) {  
    MessageHeaders headers = message.getHeaders();  
    log.info("Received from partition {} with timestamp {}",  
            headers.get(KafkaHeaders.RECEIVED_PARTITION_ID),  
            headers.get(KafkaHeaders.RECEIVED_TIMESTAMP));  
    ui.displayOrder(order);  
}
```

It's worth noting that the message payload is also available via `ConsumerRecord.value()` or `Message.getPayload()`. This means that you could ask for the `TacoOrder` through those objects instead of asking for it directly as a parameter to `handle()`.

Summary

- Asynchronous messaging provides a layer of indirection between communicating applications, which allows for looser coupling and greater scalability.
- Spring supports asynchronous messaging with JMS, RabbitMQ, or Apache Kafka.
- Applications can use template-based clients (`JmsTemplate`, `RabbitTemplate`, or `KafkaTemplate`) to send messages via a message broker.
- Receiving applications can consume messages in a pull-based model using the same template-based clients.
- Messages can also be pushed to consumers by applying message listener annotations (`@JmsListener`, `@RabbitListener`, or `@KafkaListener`) to bean methods.

10

Integrating Spring

This chapter covers

- Processing data in real time
- Defining integration flows
- Using Spring Integration's Java DSL definition
- Integrating with emails, filesystems, and other external systems

One of the most frustrating things I encounter as I travel is being on a long flight and having a poor or nonexistent in-flight internet connection. I like to use my air time to get some work done, including writing many of the pages of this book. If there's no network connection, I'm at a disadvantage if I need to fetch a library or look up a Javadoc, and I'm not able to get much work done. I've learned to pack a book to read for those occasions.

Just as we need to connect to the internet to be productive, many applications must connect to external systems to perform their work. An application may need to read or send emails, interact with an external API, or react to data being written to a database. And, as data is ingested from or written to these external systems, the application may need to process data in some way to translate it to or from the application's own domain.

In this chapter, you'll see how to employ common integration patterns with Spring Integration. Spring Integration is a ready-to-use implementation of many of the integration patterns that are catalogued in *Enterprise Integration Patterns* by Gregor Hohpe and Bobby Woolf (Addison-Wesley, 2003). Each pattern is implemented as a component through which messages ferry data in a pipeline. Using Spring configuration, you can assemble these components into a pipeline through which data flows. Let's get started by defining a simple integration flow that introduces many of the features and characteristics of working with Spring Integration.

10.1 *Declaring a simple integration flow*

Generally speaking, Spring Integration enables the creation of integration flows through which an application can receive or send data to some resource external to the application itself. One such resource that an application may integrate with is the filesystem. Therefore, among Spring Integration's many components are channel adapters for reading and writing files.

To get your feet wet with Spring Integration, you're going to create an integration flow that writes data to the filesystem. To get started, you need to add Spring Integration to your project build. For Maven, the necessary dependencies follow:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-integration</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.integration</groupId>
  <artifactId>spring-integration-file</artifactId>
</dependency>
```

The first dependency is the Spring Boot starter for Spring Integration. This dependency is essential to developing a Spring Integration flow, regardless of what the flow may integrate with. Like all Spring Boot starter dependencies, it's available as a checkbox in the Initializr¹ form.

The second dependency is for Spring Integration's file endpoint module. This module is one of over two dozen endpoint modules used to integrate with external systems. We'll talk more about the endpoint modules in section 10.2.9. But, for now, know that the file endpoint module offers the ability to ingest files from the filesystem into an integration flow and/or to write data from a flow to the filesystem.

Next you need to create a way for the application to send data into an integration flow so that it can be written to a file. To do that, you'll create a gateway interface, such as the one shown next.

¹ See <https://start.spring.io/>.

Listing 10.1 Message gateway interface to transform method invocations into messages

```
package sia6;

import org.springframework.integration.annotation.MessagingGateway;
import org.springframework.integration.file.FileHeaders;
import org.springframework.messaging.handler.annotation.Header;

@MessagingGateway(defaultRequestChannel="textInChannel")
public interface FileWriterGateway {

    void writeToFile(
        @Header(FileHeaders.FILENAME) String filename,
        String data);
}
```



Although it's a simple Java interface, there's a lot to be said about `FileWriterGateway`. The first thing you'll notice is that it's annotated with `@MessagingGateway`. This annotation tells Spring Integration to generate an implementation of this interface at run time—similar to how Spring Data automatically generates implementations of repository interfaces. Other parts of the code will use this interface when they need to write a file.

The `defaultRequestChannel` attribute of `@MessagingGateway` indicates that any messages resulting from a call to the interface methods should be sent to the given message channel. In this case, you state that any messages that result from a call to `writeToFile()` should be sent to the channel whose name is `textInChannel`.

As for the `writeToFile()` method, it accepts a filename as a `String`, and another `String` that will contain the text should be written to a file. What's notable about this method signature is that the `filename` parameter is annotated with `@Header`. In this case, the `@Header` annotation indicates that the value passed to `filename` should be placed in a message header (specified as `FileHeaders.FILENAME`, which is a constant in the `FileHeaders` class that is equal to the value `"file_name"`) rather than in the message payload. The data parameter value, on the other hand, is carried in the message payload.

Now that you've created a message gateway, you need to configure the integration flow. Although the Spring Integration starter dependency that you added to your build enables essential autoconfiguration for Spring Integration, it's still up to you to write additional configurations to define flows that meet the needs of the application. Three configuration options for declaring integration flows follow:

- XML configuration
- Java configuration
- Java configuration with a DSL

We'll take a look at all three of these configuration styles for Spring Integration, starting with the old-timer, XML configuration.

10.1.1 Defining integration flows with XML

Although I've avoided using XML configuration in this book, Spring Integration has a long history of integration flows defined in XML. Therefore, I think it's worthwhile for me to show at least one example of an XML-defined integration flow. The following listing shows how to configure your sample flow in XML.

Listing 10.2 Defining an integration flow with Spring XML configuration

```
<?xml version="1.0" encoding="UTF-8"?><beans
  xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:int="http://www.springframework.org/schema/integration"
  xmlns:int-file="http://www.springframework.org/schema/integration/file"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/integration
    http://www.springframework.org/schema/integration/spring-integration.xsd
    http://www.springframework.org/schema/integration/file
    http://www.springframework.org/schema/integration/file/spring-
      integration-file.xsd">

  <int:channel id="textInChannel" />
  <int:transformer id="upperCase"
    input-channel="textInChannel"
    output-channel="fileWriterChannel"
    expression="payload.toUpperCase()" />
  <int:channel id="fileWriterChannel" />
  <int-file:outbound-channel-adapter id="writer"
    channel="fileWriterChannel"
    directory="/tmp/sia6/files"
    mode="APPEND"
    append-new-line="true" />
</beans>
```

← Declares textInChannel

← Transforms the text

← Declares fileWriterChannel

← Writes the text to a file

Breaking down the XML in listing 10.2, we get the following:

- You configured a channel named `textInChannel`. You'll recognize this as the same channel that's set as the request channel for `FileWriterGateway`. When the `writeToFile()` method is called on `FileWriterGateway`, the resulting message is published to this channel.
- You configured a transformer that receives messages from `textInChannel`. It uses a Spring Expression Language (SpEL) expression to call `toUpperCase()` on the message payload. The result of the uppercase operation is then published to `fileWriterChannel`.
- You configured the channel named `fileWriterChannel`. This channel serves as the conduit that connects the transformer with the outbound channel adapter.

- Finally, you configured an outbound channel adapter using the `int-file` namespace. This XML namespace is provided by Spring Integration's file module to write files. As you configured it, it receives messages from `fileWriter-Channel` and writes the message payload to a file whose name is specified in the message's "file_name" header in the directory specified in the directory attribute. If the file already exists, the file will be appended with a newline rather than be overwritten.

This flow is illustrated in figure 10.1 using graphical elements styled after those in *Enterprise Integration Patterns*.

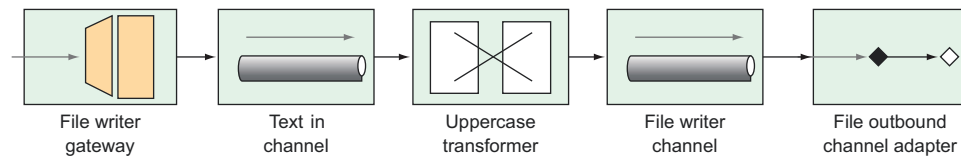


Figure 10.1 The file writer integration flow

The flow comprises five components: a gateway, two channels, a transformer, and a channel adapter. These are just a few of the components that can be assembled into an integration flow. We'll explore these components and others supported by Spring Integration in section 10.2.

If you want to use XML configuration in a Spring Boot application, you'll need to import the XML as a resource into the Spring application. The easiest way to do this is to use Spring's `@ImportResource` annotation, shown in the next code sample, on one of your application's Java configuration classes:

```

@Configuration
@ImportResource("classpath:/filewriter-config.xml")
public class FileWriterIntegrationConfig { ... }
  
```

Although XML-based configuration has served Spring Integration well, most developers have grown wary of using XML. (And, as I said, I'm avoiding XML configuration in this book.) Let's set aside those angle brackets and turn our attention to Spring Integration's Java configuration style.

10.1.2 Configuring integration flows in Java

Most modern Spring applications have eschewed XML configuration in favor of Java configuration. In fact, in Spring Boot applications, Java configuration is a natural style to complement autoconfiguration. Therefore, if you're adding an integration flow to a Spring Boot application, it makes perfect sense to define the flow in Java.

As a sample of how to write an integration flow with Java configuration, take a look at the next listing. This shows the same file-writing integration flow as before, but this time it's written in Java.

Listing 10.3 Using Java configuration to define an integration flow

```
package sia6;

import java.io.File;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.annotation.ServiceActivator;
import org.springframework.integration.annotation.Transformer;
import org.springframework.integration.file.FileWritingMessageHandler;
import org.springframework.integration.file.support.FileExistsMode;
import org.springframework.integration.transformer.GenericTransformer;

@Configuration
public class FileWriterIntegrationConfig {

    @Bean
    @Transformer(inputChannel="textInChannel",
                 outputChannel="fileWriterChannel")
    public GenericTransformer<String, String> upperCaseTransformer() {
        return text -> text.toUpperCase();
    }

    @Bean
    @ServiceActivator(inputChannel="fileWriterChannel")
    public FileWritingMessageHandler fileWriter() {
        FileWritingMessageHandler handler =
            new FileWritingMessageHandler(new File("/tmp/sia6/files"));
        handler.setExpectReply(false);
        handler.setFileExistsMode(FileExistsMode.APPEND);
        handler.setAppendNewLine(true);
        return handler;
    }
}
```

Declares a
transformer

Declares a
file writer

With Java configuration, you declare two beans: a transformer and a file-writing message handler. The transformer is a `GenericTransformer`. Because `GenericTransformer` is a functional interface, you're able to provide its implementation as a lambda that calls `toUpperCase()` on the message text. The transformer bean is annotated with `@Transformer`, designating it as a transformer in the integration flow that receives messages on a channel named `textInChannel` and writes messages to the channel named `fileWriterChannel`.

As for the file-writing bean, it's annotated with `@ServiceActivator` to indicate that it'll accept messages from `fileWriterChannel` and hand those messages over to the service defined by an instance of `FileWritingMessageHandler`. `FileWritingMessageHandler` is a message handler that writes a message payload to a file in a specified

directory using a filename specified in the message’s “file_name” header. As with the XML example, `FileWritingMessageHandler` is configured to append to the file with a newline.

One thing unique about the configuration of the `FileWritingMessageHandler` bean is that there’s a call to `setExpectReply(false)` to indicate that the service activator shouldn’t expect a reply channel (a channel through which a value may be returned to upstream components in the flow). If you don’t call `setExpectReply (false)`, the file-writing bean defaults to `true`, and, although the pipeline still functions as expected, you’ll see a few errors logged stating that no reply channel was configured.

You’ll also notice that you didn’t need to explicitly declare the channels. The `textInChannel` and `fileWriterChannel` will be created automatically if no beans with those names exist. But if you want more control over how the channels are configured, you can explicitly construct them as beans like this:

```
@Bean
public MessageChannel textInChannel() {
    return new DirectChannel();
}
...
@Bean
public MessageChannel fileWriterChannel() {
    return new DirectChannel();
}
```

The Java configuration option is arguably easier to read—and slightly briefer—and is certainly consistent with the Java-only configuration I’m shooting for in this book. But it can be made even more streamlined with Spring Integration’s Java DSL (domain-specific language) configuration style.

10.1.3 Using Spring Integration’s DSL configuration

Let’s take one more stab at defining the file-writing integration flow. This time, you’ll still define it in Java, but you’ll use Spring Integration’s Java DSL. Rather than declare an individual bean for each component in the flow, you’ll declare a single bean that defines the entire flow.

Listing 10.4 Providing a fluent API for designing integration flows

```
package sia6;

import java.io.File;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.dsl.IntegrationFlow;
import org.springframework.integration.dsl.IntegrationFlows;
import org.springframework.integration.dsl.MessageChannels;
import org.springframework.integration.file.dsl.Files;
import org.springframework.integration.file.support.FileExistsMode;
```

```

@Configuration
public class FileWriterIntegrationConfig {

    @Bean
    public IntegrationFlow fileWriterFlow() {
        return IntegrationFlows
            .from(MessageChannels.direct("textInChannel"))
            .<String, String>transform(t -> t.toUpperCase())
            .handle(Files
                .outboundAdapter(new File("/tmp/sia6/files"))
                .fileExistsMode(FileExistsMode.APPEND)
                .appendNewLine(true))
            .get();
    }
}

```

Inbound channel

Declares a transformer

Handles writing to a file

This new configuration is as terse as it can possibly be, capturing the entire flow in a single bean method. The `IntegrationFlows` class initiates the builder API, from which you can declare the flow.

In listing 10.4, you start by receiving messages from the channel named `textInChannel`, which then go to a transformer that uppercases the message payload. After the transformer, messages are handled by an outbound channel adapter created from the `Files` type provided in Spring Integration's file module. Finally, a call to `get()` builds the `IntegrationFlow` to be returned. In short, this single bean method defines the same integration flow as the XML and Java configuration examples.

You'll notice that, as with the Java configuration example, you don't need to explicitly declare channel beans. Although you reference `textInChannel`, it's automatically created by Spring Integration because there's no existing channel bean with that name. But you can explicitly declare the channel bean if you want.

As for the channel that connects the transformer to the outbound channel adapter, you don't even reference it by name. If there's a need to explicitly configure the channel, you can reference it by name in the flow definition with a call to `channel()` as follows:

```

@Bean
public IntegrationFlow fileWriterFlow() {
    return IntegrationFlows
        .from(MessageChannels.direct("textInChannel"))
        .<String, String>transform(t -> t.toUpperCase())
        .channel(MessageChannels.direct("FileWriterChannel"))
        .handle(Files
            .outboundAdapter(new File("/tmp/sia6/files"))
            .fileExistsMode(FileExistsMode.APPEND)
            .appendNewLine(true))
        .get();
}

```

One thing to keep in mind when working with Spring Integration’s Java DSL (as with any fluent API) is that you must employ whitespace shrewdly to maintain readability. In the example given here, I’ve been careful to indent lines to indicate blocks of related code. For even longer, more complex flows, you may even consider extracting portions of the flow into separate methods or subflows for better readability.

Now that you’ve seen a simple flow defined using three different configuration styles, let’s step back and take a look at Spring Integration’s big picture.

10.2 Surveying the Spring Integration landscape

Spring Integration covers a lot of ground with a multitude of integration scenarios. Trying to include all of it in a single chapter would be like trying to fit an elephant in an envelope. Instead of a comprehensive treatment of Spring Integration, I’ll present a photograph of the Spring Integration elephant to give you some idea of how it works. Then you’ll create one more integration flow that adds functionality to the Taco Cloud application.

An integration flow is composed of one or more of the following components. Before you write any more code, we’ll take a brief look at the role each of these components plays in an integration flow:

- *Channel*—Passes messages from one element to another
- *Filter*—Conditionally allows messages to pass through the flow based on some criteria
- *Transformer*—Changes message values and/or converts message payloads from one type to another
- *Router*—Directs messages to one of several channels, typically based on message headers
- *Splitter*—Splits incoming messages into two or more messages, each sent to different channels
- *Aggregator*—The opposite of a splitter; combines multiple messages coming in from separate channels into a single message
- *Service activator*—Hands a message off to some Java method for processing, and then publishes the return value on an output channel
- *Channel adapter*—Connects a channel to some external system or transport; can either accept input or write to the external system
- *Gateway*—Passes data into an integration flow via an interface

You’ve already seen a few of these components in play when you defined the file-writing integration flow. The `FileWriterGateway` interface was the gateway through which an application submitted text to be written to a file. You also defined a transformer to convert the given text to uppercase; then you declared a service gateway that performed the task of writing the text to a file. And the flow had two channels, `textInChannel` and `fileWriterChannel`, that connected the other components with each other. Now, a quick tour of the integration flow components, as promised.

10.2.1 Message channels

Message channels are the means by which messages move through an integration pipeline, as shown in figure 10.2. They're the pipes that connect all the other parts of Spring Integration plumbing together.

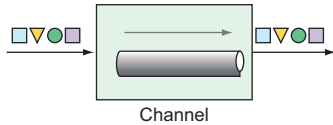


Figure 10.2 Message channels are conduits through which data flows between other components in an integration flow.

Spring Integration provides several channel implementations, including the following:

- *PublishSubscribeChannel*—Messages published into a *PublishSubscribeChannel* are passed on to one or more consumers. If multiple consumers exist, all of them receive the message.
- *QueueChannel*—Messages published into a *QueueChannel* are stored in a queue until pulled by a consumer in a first in, first out (FIFO) fashion. If multiple consumers exist, only one of them receives the message.
- *PriorityChannel*—Like *QueueChannel* but, rather than FIFO behavior, messages are pulled by consumers based on the message priority header.
- *RendezvousChannel*—Like *QueueChannel* except that the sender blocks the channel until a consumer receives the message, effectively synchronizing the sender with the consumer.
- *DirectChannel*—Like *PublishSubscribeChannel*, but sends a message to a single consumer by invoking the consumer in the same thread as the sender. This allows for transactions to span across the channel.
- *ExecutorChannel*—Similar to *DirectChannel*, but the message dispatch occurs via a *TaskExecutor*, taking place in a thread separate from the sender. This channel type doesn't support transactions that span the channel.
- *FluxMessageChannel*—A Reactive Streams Publisher message channel based on Project Reactor's *Flux*. (We'll talk more about Reactive Streams, Reactor, and *Flux* in chapter 11.)

In both the Java configuration and Java DSL styles, input channels are automatically created, with *DirectChannel* as the default. But if you want to use a different channel implementation, you'll need to explicitly declare the channel as a bean and reference it in the integration flow. For example, to declare a *PublishSubscribeChannel*, you'd declare the following *@Bean* method:

```
@Bean
public MessageChannel orderChannel() {
    return new PublishSubscribeChannel();
}
```

Then you'd reference this channel by name in the integration flow definition. For example, if the channel were being consumed by a service activator bean, you'd reference it in the `inputChannel` attribute of `@ServiceActivator` like so:

```
@ServiceActivator(inputChannel="orderChannel")
```

Or, if you're using the Java DSL configuration style, you'd reference it with a call to `channel()` as follows:

```
@Bean
public IntegrationFlow orderFlow() {
    return IntegrationFlows
        ...
        .channel("orderChannel")
        ...
        .get();
}
```

It's important to note that if you're using `QueueChannel`, the consumers must be configured with a poller. For instance, suppose that you've declared a `QueueChannel` bean like this:

```
@Bean
public MessageChannel orderChannel() {
    return new QueueChannel();
}
```

You'd need to make sure that the consumer is configured to poll the channel for messages. In the case of a service activator, the `@ServiceActivator` annotation might look like this:

```
@ServiceActivator(inputChannel="orderChannel",
    poller=@Poller(fixedRate="1000"))
```

In this example, the service activator polls from the channel named `orderChannel` every 1 second (or 1,000 ms).

10.2.2 Filters

You can place filters in the midst of an integration pipeline to allow or disallow messages from proceeding to the next step in the flow, as shown in figure 10.3.

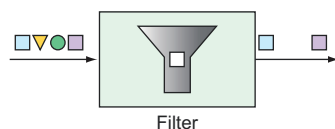


Figure 10.3 Filters based on some criteria allow or disallow messages from proceeding in the pipeline.

For example, suppose that messages containing integer values are published through a channel named `numberChannel`, but you want only even numbers to pass on to the channel named `evenNumberChannel`. In that case, you could declare a filter with the `@Filter` annotation like this:

```
@Filter(inputChannel="numberChannel",
        outputChannel="evenNumberChannel")
public boolean evenNumberFilter(Integer number) {
    return number % 2 == 0;
}
```

Alternatively, if you're using the Java DSL configuration style to define your integration flow, you could make a call to `filter()` like this:

```
@Bean
public IntegrationFlow evenNumberFlow(AtomicInteger integerSource) {
    return IntegrationFlows
        ...
        .<Integer>filter((p) -> p % 2 == 0)
        ...
        .get();
}
```

In this case, you use a lambda to implement the filter. But, in truth, the `filter()` method accepts a `GenericSelector` as an argument. This means that you can implement the `GenericSelector` interface instead, should your filtering needs be too involved for a simple lambda.

10.2.3 Transformers

Transformers perform some operation on messages, typically resulting in a different message and, possibly, with a different payload type (see figure 10.4). The transformation can be something simple, such as performing mathematic operations on a number or manipulating a `String` value. Or the transformation can be more complex, such as using a `String` value representing an ISBN to look up and return details of the corresponding book.

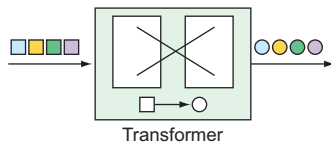


Figure 10.4 Transformers morph messages as they flow through an integration flow.

For example, suppose that integer values are being published on a channel named `numberChannel`, and you want to convert those numbers to a `String` containing the Roman numeral equivalent. In that case, you can declare a bean of type `GenericTransformer` and annotate it with `@Transformer` as follows:

```

@Bean
@Transformer(inputChannel="numberChannel",
             outputChannel="romanNumberChannel")
public GenericTransformer<Integer, String> romanNumTransformer() {
    return RomanNumbers::toRoman;
}

```

The `@Transformer` annotation designates this bean as a transformer bean that receives `Integer` values from the channel named `numberChannel` and uses a static method named `toRoman()` to do the conversion. (The `toRoman()` method is statically defined in a class named `RomanNumbers` and referenced here with a method reference.) The result is published to the channel named `romanNumberChannel`.

In the Java DSL configuration style, it's even easier with a call to `transform()`, passing in the method reference to the `toRoman()` method as follows:

```

@Bean
public IntegrationFlow transformerFlow() {
    return IntegrationFlows
        ...
        .transform(RomanNumbers::toRoman)
        ...
        .get();
}

```

Although you've used a method reference in both of the transformer code samples, know that the transformer can also be specified as a lambda. Or, if the transformer is complex enough to warrant a separate Java class, you can inject it as a bean into the flow configuration and pass the reference to the `transform()` method like so:

```

@Bean
public RomanNumberTransformer romanNumberTransformer() {
    return new RomanNumberTransformer();
}

@Bean
public IntegrationFlow transformerFlow(
    RomanNumberTransformer romanNumberTransformer) {
    return IntegrationFlows
        ...
        .transform(romanNumberTransformer)
        ...
        .get();
}

```

Here, you declare a bean of type `RomanNumberTransformer`, which itself is an implementation of Spring Integration's `Transformer` or `GenericTransformer` interfaces. The bean is injected into the `transformerFlow()` method and passed to the `transform()` method when defining the integration flow.

10.2.4 Routers

Routers, based on some routing criteria, allow for branching in an integration flow, directing messages to different channels (see figure 10.5).

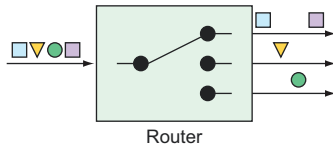


Figure 10.5 Routers direct messages to different channels, based on some criteria applied to the messages.

For example, suppose that you have a channel named `numberChannel` through which integer values flow. And let's say that you want to direct all messages with even numbers to a channel named `evenChannel`, whereas messages with odd numbers are routed to a channel named `oddChannel`. To create such a routing in your integration flow, you can declare a bean of type `AbstractMessageRouter` and annotate the bean with `@Router`, as shown next:

```
@Bean
@Router(inputChannel="numberChannel")
public AbstractMessageRouter evenOddRouter() {
    return new AbstractMessageRouter() {
        @Override
        protected Collection<MessageChannel>
            determineTargetChannels(Message<?> message) {
            Integer number = (Integer) message.getPayload();
            if (number % 2 == 0) {
                return Collections.singleton(evenChannel());
            }
            return Collections.singleton(oddChannel());
        }
    };
}

@Bean
public MessageChannel evenChannel() {
    return new DirectChannel();
}

@Bean
public MessageChannel oddChannel() {
    return new DirectChannel();
}
```

The `AbstractMessageRouter` bean declared here accepts messages from an input channel named `numberChannel`. The implementation, defined as an anonymous inner class, examines the message payload, and, if it's an even number, returns the channel named `evenChannel` (declared as a bean after the router bean). Otherwise,

the number in the channel payload must be odd, in which case, the channel named `oddChannel` is returned (also declared in a bean declaration method).

In Java DSL form, routers are declared by calling `route()` in the course of a flow definition, as shown here:

```
@Bean
public IntegrationFlow numberRoutingFlow(AtomicInteger source) {
    return IntegrationFlows
        ...
        .<Integer, String>route(n -> n%2==0 ? "EVEN":"ODD", mapping -> mapping
            .subFlowMapping("EVEN", sf -> sf
                .<Integer, Integer>transform(n -> n * 10)
                .handle((i,h) -> { ... })
            )
            .subFlowMapping("ODD", sf -> sf
                .transform(RomanNumbers::toRoman)
                .handle((i,h) -> { ... })
            )
        )
        .get();
}
```

Although it's still possible to declare an `AbstractMessageRouter` and pass it into `route()`, this example uses a lambda to determine whether a message payload is odd or even. If it's even, then a `String` value of `EVEN` is returned. If it's odd, then `ODD` is returned. These values are then used to determine which submapping will handle the message.

10.2.5 Splitters

At times in an integration flow, it can be useful to split a message into multiple messages to be handled independently. Splitters, as illustrated in figure 10.6, will split and handle those messages for you.

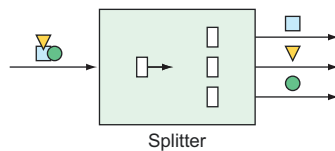


Figure 10.6 Splitters break down messages into two or more separate messages that can be handled by separate subflows.

Splitters are useful in many circumstances, but you might use a splitter for the following two essential use cases:

- *A message payload contains a collection of items of the same type that you'd like to process as individual message payloads.* For example, a message carrying a list of products might be split into multiple messages with payloads of one product each.
- *A message payload carries information that, although related, can be split into two or more messages of different types.* For example, a purchase order might carry delivery,

billing, and line-item information. The delivery details might be processed by one subflow, billing by another, and line items in yet another. In this use case, the splitter is typically followed by a router that routes messages by payload type to ensure that the data is handled by the right subflow.

When splitting a message payload into two or more messages of different types, it's usually sufficient to define a POJO that extracts the individual pieces of the incoming payload and returns them as elements of a collection.

For example, suppose that you want to split a message carrying a purchase order into two messages: one carrying the billing information and another carrying a list of line items. The following `OrderSplitter` will do the job:

```
public class OrderSplitter {
    public Collection<Object> splitOrderIntoParts(PurchaseOrder po) {
        ArrayList<Object> parts = new ArrayList<>();
        parts.add(po.getBillingInfo());
        parts.add(po.getLineItems());
        return parts;
    }
}
```

You can then declare an `OrderSplitter` bean as part of the integration flow by annotating it with `@Splitter` like this:

```
@Bean
@Splitter(inputChannel="poChannel",
          outputChannel="splitOrderChannel")
public OrderSplitter orderSplitter() {
    return new OrderSplitter();
}
```

Here, purchase orders arrive on the channel named `poChannel` and are split by `OrderSplitter`. Then, each item in the returned collection is published as a separate message in the integration flow to a channel named `splitOrderChannel`. At this point in the flow, you can declare a `PayloadTypeRouter` to route the billing information and the line items to their own subflow as follows:

```
@Bean
@Router(inputChannel="splitOrderChannel")
public MessageRouter splitOrderRouter() {
    PayloadTypeRouter router = new PayloadTypeRouter();
    router.setChannelMapping(
        BillingInfo.class.getName(), "billingInfoChannel");
    router.setChannelMapping(
        List.class.getName(), "lineItemsChannel");
    return router;
}
```

As its name implies, `PayloadTypeRouter` routes messages to different channels based on their payload type. As configured here, messages whose payload is of type `BillingInfo`

are routed to a channel named `billingInfoChannel` for further processing. As for the line items, they're in a `java.util.List` collection; therefore, you map payloads of type `List` to be routed to a channel named `lineItemsChannel`.

As things stand, the flow splits into two subflows: one through which `BillingInfo` objects flow and another through which a `List<LineItem>` flows. But what if you want to break it down further such that instead of dealing with a `List` of `LineItem` objects, you process each `LineItem` separately? All you need to do to split the line-item list into multiple messages, one for each line item, is write a method (not a bean) that's annotated with `@Splitter` and returns a collection of `LineItem` objects, perhaps something like this:

```
@Splitter(inputChannel="lineItemsChannel", outputChannel="lineItemChannel")
public List<LineItem> lineItemSplitter(List<LineItem> lineItems) {
    return lineItems;
}
```

When a message carrying a payload of `List<LineItem>` arrives in the channel named `lineItemsChannel`, it passes into the `lineItemSplitter()` method. Per the rules of a splitter, the method must return a collection of the items to be split. In this case, you already have a collection of `LineItem` objects, so you just return the collection directly. As a result, each `LineItem` in the collection is published in a message of its own to the channel named `lineItemChannel`.

If you'd rather use the Java DSL to declare the same splitter/router configuration, you can do so with calls to `split()` and `route()` as shown here:

```
return IntegrationFlows
    ...
    .split(orderSplitter())
    .<Object, String> route(
        p -> {
            if (p.getClass().isAssignableFrom(BillingInfo.class)) {
                return "BILLING_INFO";
            } else {
                return "LINE_ITEMS";
            }
        }, mapping -> mapping
        .subFlowMapping("BILLING_INFO", sf -> sf
            .<BillingInfo> handle((billingInfo, h) -> {
                ...
            })))
        .subFlowMapping("LINE_ITEMS", sf -> sf
            .split()
            .<LineItem> handle((lineItem, h) -> {
                ...
            })))
    )
    .get();
```

The DSL form of the flow definition is certainly terser, if not arguably a bit more difficult to follow. We could clean this up a bit by extracting the lambdas to methods. For

example, we could use the following three methods to replace the lambdas used in the flow definition:

```
private String route(Object p) {
    return p.getClass().isAssignableFrom(BillingInfo.class)
        ? "BILLING_INFO"
        : "LINE_ITEMS";
}

private BillingInfo handleBillingInfo(
    BillingInfo billingInfo, MessageHeaders h) {
    // ...
}

private LineItem handleLineItems(
    LineItem lineItem, MessageHeaders h) {
    // ...
}
```

Then, we could rewrite the integration flow with method references like this:

```
return IntegrationFlows
    ...
    .split()
    .route(
        this::route,
        mapping -> mapping
            .subFlowMapping("BILLING_INFO", sf -> sf
                .<BillingInfo> handle(this::handleBillingInfo))
            .subFlowMapping("LINE_ITEMS", sf -> sf
                .split()
                .<LineItem> handle(this::handleLineItems)));
```

Either way, this uses the same `OrderSplitter` to split the order as the Java configuration example. After the order is split, it's routed by its type to two separate subflows.

10.2.6 Service activators

Service activators receive messages from an input channel and send those messages to an implementation of `MessageHandler`, as shown in figure 10.7.

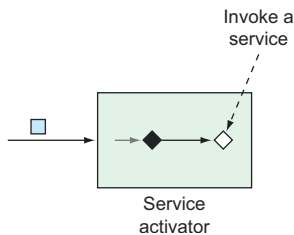


Figure 10.7 Service activators invoke some service by way of a `MessageHandler` on receipt of a message.

Spring Integration offers several `MessageHandler` implementations out of the box (even `PayloadTypeRouter` is an implementation of `MessageHandler`), but you'll often need to provide some custom implementation to act as a service activator. As an example, the following code shows how to declare a `MessageHandler` bean, configured to be a service activator:

```
@Bean
@ServiceActivator(inputChannel="someChannel")
public MessageHandler sysoutHandler() {
    return message -> {
        System.out.println("Message payload: " + message.getPayload());
    };
}
```

The bean is annotated with `@ServiceActivator` to designate it as a service activator that handles messages from the channel named `someChannel`. As for the `MessageHandler` itself, it's implemented via a lambda. Although it's a simple `MessageHandler`, when given a `Message`, it emits its payload to the standard output stream.

Alternatively, you could declare a service activator that processes the data in the incoming message before returning a new payload, as shown in the next code snippet. In that case, the bean should be a `GenericHandler` rather than a `MessageHandler`.

```
@Bean
@ServiceActivator(inputChannel="orderChannel",
                  outputChannel="completeChannel")
public GenericHandler<EmailOrder> orderHandler(
    OrderRepository orderRepo) {
    return (payload, headers) -> {
        return orderRepo.save(payload);
    };
}
```

In this case, the service activator is a `GenericHandler` that expects messages with a payload of type `EmailOrder`. When the order arrives, it's saved via a repository; the resulting saved `EmailOrder` is returned to be sent to the output channel whose name is `completeChannel`.

You may notice that a `GenericHandler` is given not only the payload but also the message headers (even if the example doesn't use those headers in any way). If you prefer, you can also use service activators in the Java DSL configuration style by passing a `MessageHandler` or `GenericHandler` to `handle()` in the flow definition as follows:

```
public IntegrationFlow someFlow() {
    return IntegrationFlows
        ...
        .handle(msg -> {
            System.out.println("Message payload: " + msg.getPayload());
        })
        .get();
}
```


In this case, the `MessageHandler` is given as a lambda, but you could also provide it as a method reference or even as an instance of a class that implements the `MessageHandler` interface. If you give it a lambda or method reference, be aware that it accepts a message as a parameter.

Similarly, `handle()` can be written to accept a `GenericHandler` if the service activator isn't intended to be the end of the flow. Applying the order-saving service activator from before, you could configure the flow with the Java DSL like this:

```
public IntegrationFlow orderFlow(OrderRepository orderRepo) {
    return IntegrationFlows
        ...
        .<EmailOrder>handle((payload, headers) -> {
            return orderRepo.save(payload);
        })
        ...
        .get();
}
```

When working with a `GenericHandler`, the lambda or method reference accepts the message payload and headers as parameters. Also, if you choose to use `GenericHandler` at the end of a flow, you'll need to return null, or else you'll get errors indicating that there's no output channel specified.

10.2.7 Gateways

Gateways are the means by which an application can submit data into an integration flow and, optionally, receive a response that's the result of the flow. Implemented by Spring Integration, gateways are realized as interfaces that the application can call to send messages to the integration flow (see figure 10.8).

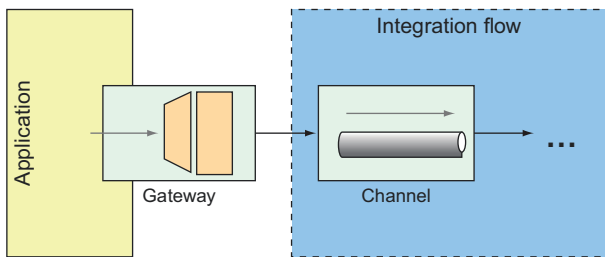


Figure 10.8 Service gateways are interfaces through which an application can submit messages to an integration flow.

You've already seen an example of a message gateway with `FileWriterGateway`. `FileWriterGateway` was a one-way gateway with a method accepting a `String` to write to a file, returning `void`. It's just about as easy to write a two-way gateway. When writing the gateway interface, be sure that the method returns some value to publish into the integration flow.

As an example, imagine a gateway that fronts a simple integration flow that accepts a `String` and translates the given `String` to all uppercase. The gateway interface might look something like this:

```
package sia6;
import org.springframework.integration.annotation.MessagingGateway;
import org.springframework.stereotype.Component;

@Component
@MessagingGateway(defaultRequestChannel="inChannel",
                  defaultReplyChannel="outChannel")
public interface UpperCaseGateway {
    String uppercase(String in);
}
```

What's amazing about this interface is that it's not necessary to implement it. Spring Integration automatically provides an implementation at run time that sends and receives data through the specified channels.

When `uppercase()` is called, the given `String` is published to the integration flow into the channel named `inChannel`. Regardless of how the flow is defined or what it does, when data arrives in the channel named `outChannel`, it's returned from the `uppercase()` method.

As for the uppercase integration flow, it's a simplistic integration flow with only a single step to transform the `String` to uppercase. Here, it's expressed in the Java DSL configuration:

```
@Bean
public IntegrationFlow uppercaseFlow() {
    return IntegrationFlows
        .from("inChannel")
        .<String, String> transform(s -> s.toUpperCase())
        .channel("outChannel")
        .get();
}
```

As defined here, the flow starts with data coming into the channel named `inChannel`. The message payload is then transformed by the transformer, which is defined here as a lambda expression, to perform an uppercase operation. The resulting message is then published to the channel named `outChannel`, which is what you've declared as the reply channel for the `UpperCaseGateway` interface.

10.2.8 Channel adapters

Channel adapters represent the entry and exit points of an integration flow. Data enters an integration flow by way of an inbound channel adapter and exits an integration flow by way of an outbound channel adapter. This is illustrated in figure 10.9.

Inbound channel adapters can take many forms, depending on the source of the data they introduce into the flow. For example, you might declare an inbound channel

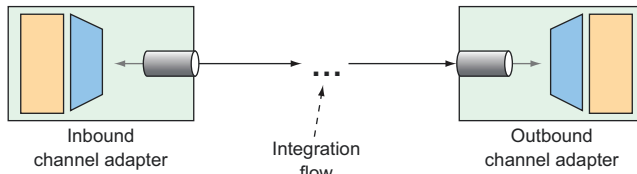


Figure 10.9 Channel adapters are the entry and exit points of an integration flow.

adapter that introduces incrementing numbers from an `AtomicInteger`² into the flow. Using Java configuration, it might look like this:

```
@Bean
@InboundChannelAdapter(
    poller=@Poller(fixedRate="1000"), channel="numberChannel")
public MessageSource<Integer> numberSource(AtomicInteger source) {
    return () -> {
        return new GenericMessage<>(source.getAndIncrement());
    };
}
```

This `@Bean` method declares an inbound channel adapter bean which, per the `@InboundChannelAdapter` annotation, submits a number from the injected `AtomicInteger` to the channel named `numberChannel` every 1 second (or 1,000 ms).

Whereas `@InboundChannelAdapter` indicates an inbound channel adapter when using Java configuration, the `from()` method is how it's done when using the Java DSL to define the integration flow. The following snippet of a flow definition shows a similar inbound channel adapter as defined in the Java DSL:

```
@Bean
public IntegrationFlow someFlow(AtomicInteger integerSource) {
    return IntegrationFlows
        .from(integerSource, "getAndIncrement",
            c -> c.poller(Pollers.fixedRate(1000)))
        ...
        .get();
}
```

Often, channel adapters are provided by one of Spring Integration's many endpoint modules. Suppose, for example, that you need an inbound channel adapter that monitors a specified directory and submits any files that are written to that directory as messages to a channel named `file-channel`. The following Java configuration uses `FileReadingMessageSource` from Spring Integration's file endpoint module to achieve that:

² `AtomicInteger` is useful for incrementing a counter in a multithreaded situation, such as the one here where multiple messages may arrive on the channel simultaneously.

```

@Bean
@InboundChannelAdapter(channel="file-channel",
                        poller=@Poller(fixedDelay="1000"))
public MessageSource<File> fileReadingMessageSource() {
    FileReadingMessageSource sourceReader = new FileReadingMessageSource();
    sourceReader.setDirectory(new File(INPUT_DIR));
    sourceReader.setFilter(new SimplePatternFileListFilter(FILE_PATTERN));
    return sourceReader;
}

```

When writing the equivalent file-reading inbound channel adapter in the Java DSL, the `inboundAdapter()` method from the `Files` class achieves the same thing. As shown next, an outbound channel adapter is the end of the line for the integration flow, handing off the final message to the application or to some other system:

```

@Bean
public IntegrationFlow fileReaderFlow() {
    return IntegrationFlows
        .from(Files.inboundAdapter(new File(INPUT_DIR))
            .patternFilter(FILE_PATTERN))
        .get();
}

```

Service activators, implemented as message handlers, often serve the purpose of an outbound channel adapter, especially when data needs to be handed off to the application itself. We've already discussed service activators, so there's no point in repeating that discussion.

It's worth noting, however, that Spring Integration endpoint modules provide useful message handlers for several common use cases. You saw an example of such an outbound channel adapter, `FileWritingMessageHandler`, in listing 10.3. Speaking of Spring Integration endpoint modules, let's take a quick look at what ready-to-use integration endpoint modules are available.

10.2.9 Endpoint modules

It's great that Spring Integration lets you create your own channel adapters. But what's even better is that Spring Integration provides more than two dozen endpoint modules containing channel adapters—both inbound and outbound—for integration with a variety of common external systems, including those listed in table 10.1.

Table 10.1 Spring Integration provides more than two dozen endpoint modules for integration with external systems.

Module	Dependency artifact ID (Group ID: <code>org.springframework.integration</code>)
AMQP	<code>spring-integration-amqp</code>
Application events	<code>spring-integration-event</code>
Atom and RSS	<code>spring-integration-feed</code>

Table 10.1 Spring Integration provides more than two dozen endpoint modules for integration with external systems. (*continued*)

Module	Dependency artifact ID (Group ID: <code>org.springframework.integration</code>)
Email	<code>spring-integration-mail</code>
Filesystem	<code>spring-integration-file</code>
FTP/FTPS	<code>spring-integration-ftp</code>
GemFire	<code>spring-integration-gemfire</code>
HTTP	<code>spring-integration-http</code>
JDBC	<code>spring-integration-jdbc</code>
JMS	<code>spring-integration-jms</code>
JMX	<code>spring-integration-jmx</code>
JPA	<code>spring-integration-jpa</code>
Kafka	<code>spring-integration-kafka</code>
MongoDB	<code>spring-integration-mongodb</code>
MQTT	<code>spring-integration-mqtt</code>
R2DBC	<code>spring-integration-r2dbc</code>
Redis	<code>spring-integration-redis</code>
RMI	<code>spring-integration-rmi</code>
RSocket	<code>spring-integration-rsocket</code>
SFTP	<code>spring-integration-sftp</code>
STOMP	<code>spring-integration-stomp</code>
Stream	<code>spring-integration-stream</code>
Syslog	<code>spring-integration-syslog</code>
TCP/UDP	<code>spring-integration-ip</code>
WebFlux	<code>spring-integration-webflux</code>
Web Services	<code>spring-integration-ws</code>
WebSocket	<code>spring-integration-websocket</code>
XMPP	<code>spring-integration-xmpp</code>
ZeroMQ	<code>spring-integration-zeromq</code>
ZooKeeper	<code>spring-integration-zookeeper</code>

One thing that's clear from looking at table 10.1 is that Spring Integration provides an extensive set of components to meet many integration needs. Most applications will

never need even a fraction of what Spring Integration offers. But it's good to know that Spring Integration has you covered if you need any of these components.

What's more, it would be impossible to cover all the channel adapters afforded by the modules listed in table 10.1 in the space of this chapter. You've already seen examples that use the filesystem module to write to the filesystem. And you're soon going to use the email module to read emails.

Each of the endpoint modules offers channel adapters that can be either declared as beans when using Java configuration or referenced via static methods when using Java DSL configuration. I encourage you to explore any of the other endpoint modules that interest you most. You'll find that they're fairly consistent in how they're used. But for now, let's turn our attention to the email endpoint module to see how you might use it in the Taco Cloud application.

10.3 *Creating an email integration flow*

You've decided that Taco Cloud should enable its customers to submit their taco designs and place orders by email. You send out flyers and place takeout ads in newspapers inviting everyone to send in their taco orders by email. It's a tremendous success! Unfortunately, it's a bit too successful. There are so many emails coming in that you have to hire temporary help to do nothing more than read all the emails and submit order details into the ordering system.

In this section, you'll implement an integration flow that polls the Taco Cloud inbox for taco order emails, parses the emails for order details, and submits the orders to Taco Cloud for handling. In short, the integration flow you're going to need will use an inbound channel adapter from the email endpoint module to ingest emails from the Taco Cloud inbox into the integration flow.

The next step in the integration flow will parse the emails into order objects that are handed off to another handler to submit orders to Taco Cloud's REST API, where they'll be processed the same as any order. To start with, let's define a simple configuration properties class to capture the specifics of how to handle Taco Cloud emails, as shown here:

```
package tacos.email;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
import lombok.Data;

@Data
@ConfigurationProperties(prefix="tacocloud.email")
@Component
public class EmailProperties {

    private String username;
    private String password;
    private String host;
    private String mailbox;
    private long pollRate = 30000;
```

```

public String getImapUrl() {
    return String.format("imaps://%s:%s@%s/%s",
        this.username, this.password, this.host, this.mailbox);
}
}

```

As you can see, `EmailProperties` captures properties that are used to produce an IMAP URL. The flow uses this URL to connect to the Taco Cloud email server and poll for emails. Among the properties captured are the email user’s username and password, as well as the hostname of the IMAP server, the mailbox to poll, and the rate at which the mailbox is polled (which defaults to every 30 seconds).

The `EmailProperties` class is annotated at the class level with `@ConfigurationProperties` with a prefix attribute set to `tacocloud.email`. This means that you can configure the details of consuming an email in the `application.yml` file like this:

```

tacocloud:
  email:
    host: imap.tacocloud.com
    mailbox: INBOX
    username: taco-in-flow
    password: 1L0v3T4c0s
    poll-rate: 10000

```

Of course, the email server configuration shown here is fictional. You’ll need to tweak it to match the email server details that you’ll be using.

Also, you may get an “unknown property” warning in your IDE. That’s because the IDE is looking for metadata it needs to understand what those properties mean. The warnings won’t break the actual code, and you can ignore them if you want. Or you can make them go away by adding the following dependency to your build (also available as a Spring Initializr option called “Spring Configuration Processor”):

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>

```

This dependency includes support for automatically generating metadata for custom configuration properties such as the ones we’re using to configure the email server details.

Now let’s use `EmailProperties` to configure the integration flow. The flow you’re aiming to create will look a little like figure 10.10.

You have the following two options when defining this flow:

- *Define it within the Taco Cloud application itself.* At the end of the flow, a service activator will call into the repositories you’ve defined to create the taco order.
- *Define it as a separate application.* At the end of the flow, a service activator will send a POST request to the Taco Cloud API to submit the taco order.

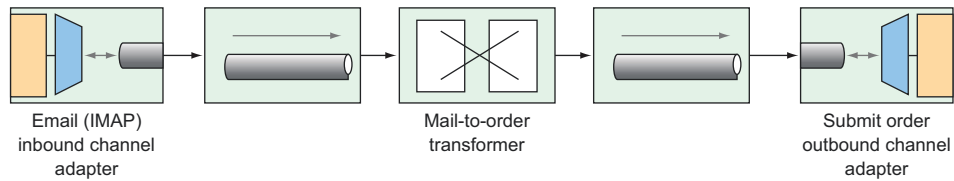


Figure 10.10 An integration flow to accept taco orders by email

Whichever you choose has little bearing on the flow itself, aside from how the service activator is implemented. But because you're going to need some types that represent tacos, orders, and ingredients, which are subtly different from those you've already defined in the main Taco Cloud application, you'll proceed by defining the integration flow in a separate application to avoid any confusion with the existing domain types.

You also have the choice of defining the flow using either XML configuration, Java configuration, or the Java DSL. I rather like the elegance of the Java DSL, so that's what you'll use. Feel free to write the flow using one of the other configuration styles if you're interested in a little extra challenge. For now, let's take a look at the Java DSL configuration for the taco order email flow as shown next.

Listing 10.5 Defining an integration flow to accept emails and submit them as orders

```

package tacos.email;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.dsl.IntegrationFlow;
import org.springframework.integration.dsl.IntegrationFlows;
import org.springframework.integration.dsl.Pollers;
import org.springframework.integration.mail.dsl.Mail;

@Configuration
public class TacoOrderEmailIntegrationConfig {

    @Bean
    public IntegrationFlow tacoOrderEmailFlow(
        EmailProperties emailProps,
        EmailToOrderTransformer emailToOrderTransformer,
        OrderSubmitMessageHandler orderSubmitHandler) {

        return IntegrationFlows
            .from(Mail.imapInboundAdapter(emailProps.getImapUrl()),
                e -> e.poller(
                    Pollers.fixedDelay(emailProps.getPollRate())))
            .transform(emailToOrderTransformer)
            .handle(orderSubmitHandler)
            .get();
    }
}

```


The taco order email flow, as defined in the `tacoOrderEmailFlow()` method, is composed of the following three distinct components:

- *An IMAP email inbound channel adapter*—This channel adapter is created with the IMAP URL generated from the `getImapUrl()` method of `EmailProperties` and polls on a delay set in the `pollRate` property of `EmailProperties`. The emails coming in are handed off to a channel connecting it to the transformer.
- *A transformer that transforms an email into an order object*—The transformer is implemented in `EmailToOrderTransformer`, which is injected into the `tacoOrderEmailFlow()` method. The orders resulting from the transformation are handed off to the final component through another channel.
- *A handler (acting as an outbound channel adapter)*—The handler accepts an order object and submits it to Taco Cloud’s REST API.

The call to `Mail.imapInboundAdapter()` is made possible by including the Email endpoint module as a dependency in your project build. The Maven dependency looks like this:

```
<dependency>
  <groupId>org.springframework.integration</groupId>
  <artifactId>spring-integration-mail</artifactId>
</dependency>
```

The `EmailToOrderTransformer` class is an implementation of Spring Integration’s `Transformer` interface, by way of extending `AbstractMailMessageTransformer` (shown in the following listing).

Listing 10.6 Converting incoming emails to taco orders using an integration transformer

```
package tacos.email;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import javax.mail.Message;
import javax.mail.MessagingException;
import javax.mail.internet.InternetAddress;
import org.apache.commons.text.similarity.LevenshteinDistance;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.integration.mail.transformer
    .AbstractMailMessageTransformer;
import org.springframework.integration.support

    .AbstractIntegrationMessageBuilder;
import org.springframework.integration.support.MessageBuilder;
import org.springframework.stereotype.Component;

@Component
public class EmailToOrderTransformer
    extends AbstractMailMessageTransformer<EmailOrder> {
```

```

private static Logger log =
    LoggerFactory.getLogger(EmailToOrderTransformer.class);

private static final String SUBJECT_KEYWORDS = "TACO ORDER";

@Override
protected AbstractIntegrationMessageBuilder<EmailOrder>
    doTransform(Message mailMessage) throws Exception {
    EmailOrder tacoOrder = processPayload(mailMessage);
    return MessageBuilder.withPayload(tacoOrder);
}

private EmailOrder processPayload(Message mailMessage) {
    try {
        String subject = mailMessage.getSubject();
        if (subject.toUpperCase().contains(SUBJECT_KEYWORDS)) {
            String email =
                ((InternetAddress) mailMessage.getFrom()[0]).getAddress();
            String content = mailMessage.getContent().toString();
            return parseEmailToOrder(email, content);
        }
    } catch (MessagingException e) {
        log.error("MessagingException: {}", e);
    } catch (IOException e) {
        log.error("IOException: {}", e);
    }
    return null;
}

private EmailOrder parseEmailToOrder(String email, String content) {
    EmailOrder order = new EmailOrder(email);
    String[] lines = content.split("\\r?\\n");
    for (String line : lines) {
        if (line.trim().length() > 0 && line.contains(":")) {
            String[] lineSplit = line.split(":");
            String tacoName = lineSplit[0].trim();
            String ingredients = lineSplit[1].trim();
            String[] ingredientsSplit = ingredients.split(",");
            List<String> ingredientCodes = new ArrayList<>();
            for (String ingredientName : ingredientsSplit) {
                String code = lookupIngredientCode(ingredientName.trim());
                if (code != null) {
                    ingredientCodes.add(code);
                }
            }

            Taco taco = new Taco(tacoName);
            taco.setIngredients(ingredientCodes);
            order.addTaco(taco);
        }
    }
    return order;
}

private String lookupIngredientCode(String ingredientName) {
    for (Ingredient ingredient : ALL_INGREDIENTS) {

```

```

        String ucIngredientName = ingredientName.toUpperCase();
        if (LevenshteinDistance.getDefaultInstance()
            .apply(ucIngredientName, ingredient.getName()) < 3 ||
            ucIngredientName.contains(ingredient.getName()) ||
            ingredient.getName().contains(ucIngredientName)) {
            return ingredient.getCode();
        }
    }
    return null;
}

private static Ingredient[] ALL_INGREDIENTS = new Ingredient[] {
    new Ingredient("FLTO", "FLOUR TORTILLA"),
    new Ingredient("COTO", "CORN TORTILLA"),
    new Ingredient("GRBF", "GROUND BEEF"),
    new Ingredient("CARN", "CARNITAS"),
    new Ingredient("TMTO", "TOMATOES"),
    new Ingredient("LETC", "LETTUCE"),
    new Ingredient("CHED", "CHEDDAR"),
    new Ingredient("JACK", "MONTERREY JACK"),
    new Ingredient("SLSA", "SALSA"),
    new Ingredient("SRCR", "SOUR CREAM")
};
}

```

`AbstractMailMessageTransformer` is a convenient base class for handling messages whose payload is an email. It takes care of extracting the email information from the incoming message into a `Message` object that's passed into the `doTransform()` method.

In the `doTransform()` method, you pass the `Message` to a private method named `processPayload()` to parse the email into an `EmailOrder` object. Although similar, the `EmailOrder` object in question isn't the same as the `TacoOrder` object used in the main Taco Cloud application; it's slightly simpler, as shown next:

```

package tacos.email;
import java.util.ArrayList;
import java.util.List;
import lombok.Data;

@Data
public class EmailOrder {

    private final String email;
    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

Rather than carry the customer's entire delivery and billing information, this `EmailOrder` class carries only the customer's email, obtained from the incoming email.

Parsing emails into taco orders is a nontrivial task. In fact, even a naive implementation involves several dozen lines of code. And those several dozen lines of code do nothing to further the discussion of Spring Integration and how to implement a transformer. Therefore, to save space, I'm leaving out the details of the `processPayload()` method.

The last thing that `EmailToOrderTransformer` does is return a `MessageBuilder` with a payload containing the `EmailOrder` object. The message that's produced by the `MessageBuilder` is sent to the final component in the integration flow: a message handler that posts the order to Taco Cloud's API. The `OrderSubmitMessageHandler`, as shown in the next listing, implements Spring Integration's `GenericHandler` to handle messages with an `EmailOrder` payload.

Listing 10.7 Posting orders to the Taco Cloud API via a message handler

```
package tacos.email;

import org.springframework.integration.handler.GenericHandler;
import org.springframework.messaging.MessageHeaders;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestTemplate;

@Component
public class OrderSubmitMessageHandler
    implements GenericHandler<EmailOrder> {

    private RestTemplate rest;
    private ApiProperties apiProps;

    public OrderSubmitMessageHandler(ApiProperties apiProps, RestTemplate rest) {
        this.apiProps = apiProps;
        this.rest = rest;
    }

    @Override
    public Object handle(EmailOrder order, MessageHeaders headers) {
        rest.postForObject(apiProps.getUrl(), order, String.class);
        return null;
    }
}
```

To satisfy the requirements of the `GenericHandler` interface, `OrderSubmitMessageHandler` overrides the `handle()` method. This method receives the incoming `EmailOrder` object and uses an injected `RestTemplate` to submit the `EmailOrder` via a POST request to the URL captured in an injected `ApiProperties` object. Finally, the `handle()` method returns `null` to indicate that this handler marks the end of the flow.

`ApiProperties` is used to avoid hardcoding the URL in the call to `postForObject()`. It's a configuration properties file that looks like this: